Lecture 04 - How to use ITensors

Joaquín Peñuela Parra

Department of Mechanical Engineering and Materials Science,
University of Pittsburgh, Pittsburgh, Pennsylvania 15261, USA

September 25, 2026

Abstract

This tutorial introduces the basic building blocks needed to work with tensors computationally: how to create, index, and manipulate them, and how operations like matrix decompositions (QR, SVD) let us compress and truncate information systematically. We'll then assemble these pieces to construct quantum states as MPS, compare them against exact diagonalization (ED) for small systems, and apply single- and two-site operators to compute physical observables such as expectation values. Along the way, we'll touch on the Hadamard product and see how MPS can even be used to efficiently encode continuous functions — a perspective that connects tensor networks to numerical analysis and quantum-inspired scientific computing. Finally, we'll briefly explore how these operations scale on GPU hardware, highlighting the performance benefits of tensor libraries for larger simulations. By the end of this tutorial, you should have the practical toolkit needed to start building and manipulating tensor network states for your own research problems.

Please fill out https://forms.gle/B51N8mB8SF3dwgBZ9
ITensor documentation: https://docs.itensor.org/Overview/

Contents

1. Preliminaries: Installation and loading **4**

2. `Index()` and `ITensor()`: How to define tensors? **4**
2.1. Defining a matrix 5
2.1.1 What does it mean that it does not care about the order? 5

3. `randomITensor()` **7**

4. `contract()` and `*`: How to contract two tensors? **8**
4.1. Product of two matrices. 8
4.2. What if we do not have common indices and use `*`? 10

5. Exercise: Define and multiply 2 tensors. **11**

6. `Array()`: How to go from an ITensor to a Julia array? **12**
6.1. **Do you have a collection of 3 matrices of size 2x4, or a collection of 4 matrices of size 2x3?** 13
6.2. Exercise: Get a collection of 2 matrices with size 4 x 3 14

7. Linear combinations of ITensors **14**

8. prime(): level of a tensor's indices **16**
8.1. How to get the common indices between different tensors? 17
8.2. Contracting only over one index even if there are two common indices 17

9. delta(): Identity or Kronecker delta tensor **19**
9.1. How to replace indices? 20
9.2. Tracing out some degrees of freedom. 21

10. Exercise: Contract over $_i$ in $A_{ij}B_{ji}$ and get a rank-1 tensor. **22**

11. Exercise: Contract over $_{i,j}$ in $A_{ij}B_{jk}C_{km}D_{mi}$ and get a rank-2 tensor. **24**

12. `combiner()`: How to mix tensor legs? **25**
12.1. Exercise: Combine i, j and k, l to have a matrix of dimensions dim(i)*dim(j) x dim(k)*dim(l). 26

13. `dag()`: How to split tensor legs? & How to get adjoint operators? **27**
13.1. Exercise: Implement $AA^\dagger$ 31

14. `qr()` and `svd()`: Descomposition of ITensors **31**
14.1. Exercise: Check that ITensors does all the mixing/splitting for us for `qr()` 33
14.2. What about the reduced SVD and the truncation error? 36
14.3. Maxdim - the maximum number of singular values (bond dimension) to keep 37
14.4. Cutoff - discards singular values whose relative contribution to the total weight falls below this threshold 38
14.5. Exercise: Calculate the truncation error 38
14.6. What if we have maxdim and cutoff at the same time? 38

15. Matrix Product States (MPS) **39**
15.1. `siteinds()`: "SiteTypes" to represent different physical degrees of freedom provided by ITensors. . . . 41
15.2. Defining states from ket notation. . . . 42
15.3. `expect()`: Calculating expectation values. . . . 43
15.4. Comparison with exact diagonalization objects 43
15.5. Exercise: Define $|0,0,0,0,0,1,1,1,1,1\rangle$ in MPS, check $\langle Z_i \rangle$ and compare with the ED object. . . . 45
15.6. Exercise: Define $|0,1,0,1,0,1,0,1,0,1\rangle$ in MPS, check $\langle Z_i \rangle$ and compare with the ED object. . . . 46
15.7. `randomMPS()` - 6:30PM 46
15.8. Addition of MPSs 47
15.9. `norm()` 47
15.10.`truncate()` 48
15.11.Exercise: What is the bond dimension used for a cutoff of 0.02? 48
15.12.`inner()`, `op()` and `apply()`: Expectation values and overlaps of MPS 48
15.13.Exercise: Define $|001100\rangle$, apply a rotation of $\theta = \pi/2$ at all sites and calculate $\langle Y_i \rangle$ 50
15.14.von-Neumann entanglement entropy 51
15.15.Encoding functions and Hadamard product (element-wise multiplication) 53
15.16.Exercise: Encode $h(x) = f^2(x)g(x)$ with $ f(x) = $\cos(x)$\$and$g(x) = \sqrt{x}e^{-0.25x}$ for $x \in [0, 20]$ using 8 qubits. . . . 58

16. GPU speed-up **60**
16.1. Moving data to the GPU 60
16.2. How to supervise memory? 61
16.3. How to clean memory? 61

1. Preliminaries: Installation and loading

To have a self-contained tutorial, we need to start with the installation. For this, you simply need to use the `Pkg` in Julia.

[2]:
```
# Please uncomment the following lines if you have not already installed␣
↪ITensors:

# using Pkg
# Pkg.add("ITensors") #Focuses on the core definition and manipulation of␣
↪individual ITensor objects, index systems, block-sparse/dense data layouts,␣
↪and basic tensor contractions.
# Pkg.add("ITensorMPS") #Focuses on tensor networks, specifically Matrix Product␣
↪States (MPS), Matrix Product Operators (MPO), and algorithms like DMRG and␣
↪TDVP. Requires ITensors.jl as a dependency.
```

[3]:
```
using ITensors
using ITensorMPS
```

It is worth mentioning that you can precompile and speed up ITensors.jl in Julia using the built-in `ITensors.compile()` function with `PackageCompiler.jl` (you probably also need to install this package it with `Pkg`).

Why? Julia uses Just-In-Time (JIT) compilation; thus, the compiler must parse, infer types, and generate native machine code before it can actually execute your function. If a function takes only a fraction of a second to run but 5 seconds to compile, your first execution will feel painfully slow.

Possible problems: `ITensors.compile()` generates an image with the compiled package inside `~/.julia/sysimages/`. This works well if you are working locally. However, the generated image can be hardware-dependent and generate some driver problems when you are running on an HPC cluster (like CRC). This is very annoying when you are using different types of GPUs.

What can we do: Well, the simplest solution (what I do) is running the same code in a very simple case before running your real code; that is enough to solve the first compilation. For example, if you want to simulate a system with 100 sites, run the code at the beginning with only 4 sites. You can also create multiple images depending on the hardware and load the specific one you need for each case.

2. `Index()` and `ITensor()`: How to define tensors?

Before we can build any tensor network, we need two basic ingredients: indices (the "legs" of a tensor) and ITensors (the tensors themselves, which carry data and know which indices they have).

- **Index()**: Defines an Index which represents a single tensor leg with a fixed dimension. It carries a unique identity (an internal ID tag), so even two indices with the same dimension are treated as distinct unless they are literally the same Index object. **This is what allows ITensor to automatically figure out how tensors should contract**.
- **Itensor()**: Defines a block of data (its storage) together with the set of Index objects labeling each dimension. Unlike a plain array, **an ITensor doesn't care about the order** in which

you list its indices

2.1 Defining a matrix

Imagine we want to define the tensor A_{ij} such as $A_{12} = 2, A_{22} = 3$ and zero everywhere else. Assume $i = \{1, 2, 3\}$ and $j = \{1, 2, 3, 4\}$.

[10]:
```
i = Index(3)
j = Index(4)

A = ITensor(i,j) #Tensor with two indexes, it will be a Matrix
A[i=>1, j => 2] = 2 #This is one way to set the elements inside the tensor (it
 ↪does not matter the order).
A[i=>2, j => 2] = 3

@show A
```

```
A = ITensor ord=2
Dim 1: (dim=3|id=150)
Dim 2: (dim=4|id=147)
NDTensors.Dense{Int64, Vector{Int64}}
 3×4
 0  2  0  0
 0  3  0  0
 0  0  0  0
```

[10]:
```
ITensor ord=2 (dim=3|id=150) (dim=4|id=147)
NDTensors.Dense{Int64, Vector{Int64}}
```

2.1.1 What does it mean that it does not care about the order?

The element A_{12} means $i = 1$ and $j = 2$, there are two ways to get it:

[11]:
```
@show A[i=>1,j=>2];
@show A[j=>2,i=>1];
@assert A[i=>1,j=>2] == A[j=>2,i=>1]
```

```
A[i => 1, j => 2] = 2
A[j => 2, i => 1] = 2
```

This is because each index has a unique internal ID tag:

[12]:
```
@show inds(A); #This is how we can get the indexes of A
```

```
inds(A) = ((dim=3|id=150), (dim=4|id=147))
```

Indeed, every time that you create an index, it will have a different tag:

[14]:
```
for k = 1:5
    i = Index(3)
```

```
    @show i
end
```

```
i = (dim=3|id=201)
i = (dim=3|id=825)
i = (dim=3|id=334)
i = (dim=3|id=581)
i = (dim=3|id=756)
```

You can also add a tag to give some information about the index:

[16]:
```
s = Index(3, "dog,spin") #A little bit random to be honest, I have never used it.
 ↪ But useful if you want to define new sites that are not predefined in␣
 ↪ITensors.
```

[16]:
```
(dim=3|id=468|"dog,spin")
```

Defining each element of the matrix is very unpractical; there is another way to do it. Imagine that you already have your matrix defined as a `Matrix{Float64}` object.

[18]:
```
array = [1.0 2.0 3.0 4.0 ; 5.0 6.0 7.0 8.0] #This is a 2x4 Matrix.

#We need to define two indices, one with dimension 2 (rows) and one with␣
 ↪dimension 4 (columns)
i = Index(4)
j = Index(2)

#Then we just need to use ITensor()

A = ITensor(array,i,j) #This is another way to give vale
@show A

#or
A = ITensor(array,j, i) #This is another way to give vale
@show A;
```

```
A = ITensor ord=2
Dim 1: (dim=4|id=916)
Dim 2: (dim=2|id=151)
NDTensors.Dense{Float64, Vector{Float64}}
 4×2
 1.0  3.0
 5.0  7.0
 2.0  4.0
 6.0  8.0

A = ITensor ord=2
Dim 1: (dim=2|id=151)
Dim 2: (dim=4|id=916)
```

```
NDTensors.Dense{Float64, Vector{Float64}}
 2×4
 1.0  2.0  3.0  4.0
 5.0  6.0  7.0  8.0
```

Both tensors contain the same information; however, when we print them, only the second one looks like our initial matrix.

[19]:
```
array
```

[19]:
```
2×4 Matrix{Float64}:
 1.0  2.0  3.0  4.0
 5.0  6.0  7.0  8.0
```

3. randomITensor()

This function creates an ITensor with normally distributed random elements instead of specific values. To be honest, in three years I have only used this function for the tutorial and maybe some debugging when I was learning how to use GPU.

[21]:
```
i = Index(2); # Index of dimension 2
j = Index(4); # Index of dimension 4
k = Index(3); # Index of dimension 3

A = randomITensor(i,j,k)
println(A)
```

```
ITensor ord=3
Dim 1: (dim=2|id=376)
Dim 2: (dim=4|id=837)
Dim 3: (dim=3|id=200)
NDTensors.Dense{Float64, Vector{Float64}}
 2×4×3
[:, :, 1] =
 -1.4650992990001035  -1.9165807599441198  ...  -0.9369949781115169
 -0.5253241292204983  -0.1267688472720655       0.8457551112844861

[:, :, 2] =
  1.342204897780929   -0.9279995423836472  ...   0.8616253966590626
 -1.5802836240817744   0.6705409100302469      -0.6690031126037048

[:, :, 3] =
  2.4443489374764273  -0.3751184884268428  ...   0.7694795827522672
 -0.5065597512310996  -0.6145048302275461      -1.37582888628385
```

4. `contract()` and $*$: How to contract two tensors?

Contraction is the fundamental operation for combining tensors in a network: any indices shared between two ITensors are automatically summed over, while unshared indices survive into the result. You don't need to specify which indices to contract — ITensor figures this out for you based on matching Index objects.

Note: `*` is the standard way to contract two ITensors. `contract()` is useful when you want to be explicit or contract a list of tensors at once (e.g. contract([A, B, C])). It behaves identically to `*` for two tensors, but is handy for writing generic code or contracting many tensors in sequence.

4.1 Product of two matrices.

Consider the tensor product: $A_{ij} * B_{ik}$ for the following matrices:

[22]:
```
A_matrix = [1.0 2.0 3.0; 4.0 5.0 6.0; 7.0 8.0 9.0]
B_matrix = [1.0 2.0 3.0; 4.0 5.0 6.0; 7.0 8.0 9.0]/10;

i = Index(3)
j = Index(3)
k = Index(3)

A = ITensor(A_matrix,i,j)
B = ITensor(B_matrix,i,k)

@show A
@show B
```

```
A = ITensor ord=2
Dim 1: (dim=3|id=204)
Dim 2: (dim=3|id=23)
NDTensors.Dense{Float64, Vector{Float64}}
 3×3
 1.0  2.0  3.0
 4.0  5.0  6.0
 7.0  8.0  9.0

B = ITensor ord=2
Dim 1: (dim=3|id=204)
Dim 2: (dim=3|id=647)
NDTensors.Dense{Float64, Vector{Float64}}
 3×3
 0.1  0.2  0.3
 0.4  0.5  0.6
 0.7  0.8  0.9
```

[22]:
```
ITensor ord=2 (dim=3|id=204) (dim=3|id=647)
NDTensors.Dense{Float64, Vector{Float64}}
```

[23]:
```
inds(A) #This is how you get the indices of A
```

[23]:
```
((dim=3|id=204), (dim=3|id=23))
```

For contraction we just need to use the function * or `contract()`

[26]:
```
@show C = A*B; #Use * is how ITensor do the contraction of the common indexes␣
↪between A and B.
@show C = B*A; #Same operation.
```

```
C = A * B = ITensor ord=2
Dim 1: (dim=3|id=23)
Dim 2: (dim=3|id=647)
NDTensors.Dense{Float64, Vector{Float64}}
 3×3
 6.6   7.800000000000001   9.0
 7.8   9.3                10.8
 9.0  10.8                12.6

C = B * A = ITensor ord=2
Dim 1: (dim=3|id=647)
Dim 2: (dim=3|id=23)
NDTensors.Dense{Float64, Vector{Float64}}
 3×3
 6.6                 7.8   9.0
 7.800000000000001   9.3  10.8
 9.0                10.8  12.6
```

[27]:
```
@show C = contract(A,B);
```

```
C = contract(A, B) = ITensor ord=2
Dim 1: (dim=3|id=23)
Dim 2: (dim=3|id=647)
NDTensors.Dense{Float64, Vector{Float64}}
 3×3
 6.6   7.800000000000001   9.0
 7.8   9.3                10.8
 9.0  10.8                12.6
```

Let's try the same without using Itensor, just Julia. In this case we we do not have an index, just matrices. We must be careful when thinking about which index we want to contract, and do the operation that we really want.

It is not just `A_matrix*B_matrix`.

[29]:
```
A_matrix*B_matrix
```

[29]:
```
3×3 Matrix{Float64}:
  3.0   3.6   4.2
  6.6   8.1   9.6
 10.2  12.6  15.0
```

We want $A_{ij} * B_{ik}$, and $(A * B)_{ik} = A_{ij} * B_{jk}$, which is a very different quantity.

So $A_{ij} * B_{ik} = A^T_{ji} * B_{ik} = (A^T * B)_{jk}$

[30]:
```
transpose(A_matrix)*B_matrix
```

[30]:
```
3×3 Matrix{Float64}:
 6.6   7.8   9.0
 7.8   9.3  10.8
 9.0  10.8  12.6
```

We got the same result, but it was harder without ITensor.

4.2 What if we do not have common indices and use *?

If we do not have indices to contract, we should expect a higher-order tensor because $A_{ij} * B_k = (AB)_{ijk}$

[32]:
```
A = randomITensor(i,j)
B = randomITensor(k)

@show AB = A*B; #Clearly it is not a matrix any more.
```

```
AB = A * B = ITensor ord=3
Dim 1: (dim=3|id=204)
Dim 2: (dim=3|id=23)
Dim 3: (dim=3|id=647)
NDTensors.Dense{Float64, Vector{Float64}}
 3×3×3
[:, :, 1] =
  0.001384846365640948    0.00016985827689261485  -0.000488623116183615
 -0.000270846059085158    0.0010171193345015785   -0.00148832246471315
 -7.075910488378745e-5   -0.0004467757318578908    0.0006846427292801331

[:, :, 2] =
  1.0456846149375185     0.12825840560602358  -0.368954772004296
 -0.20922400969760904    0.7680173527057848   -1.1238184552667851
 -0.053429542207363345  -0.3373562011804961    0.516967359340051

[:, :, 3] =
  0.5816967516261702     0.07134799235258293  -0.20524332987790694
 -0.1163877951963304     0.4272351269968526   -0.6251612919496673
 -0.029721955070318874  -0.1876655767564917    0.2875802410489086
```

5. Exercise: Define and multiply 2 tensors.

Calculate $A_{ij} * B_{jk}$ from two tensors A_{ij} and B_{jk} with $i = \{1, 2\}, j = \{1, 2, 3, 4\}$, $k = \{1, 2, 3\}$, and

- A[i=1,j=1] = 2.0
- A[1,2] = 0.7
- A[1,4] = 1.9
- A[2,2] = -3.0
- A[2,3] = -0.1
- A[2,4] = 0.5
- B[j=1,k=1] = -3.5
- B[1,2] = 0.2
- B[1,3] = 2.7
- B[2,3] = -1.0
- B[3,1] = -0.6
- B[3,3] = 4.2
- B[4,2] = -0.1
- B[4,3] = 1.7

[33]:
```
i = Index(2); # Index of dimension 2
j = Index(4); # Index of dimension 4

A = ITensor(i,j);  # Define ITensor element by element

A[i=>1,j=>1] = 2.0;
A[i=>1,j=>2] = 0.7;
A[i=>1,j=>4] = 1.9;
A[i=>2,j=>2] = -3.0;
A[i=>2,j=>3] = -0.1;
A[i=>2,j=>4] = 0.5;

B = ITensor(j,k);

B[j=>1,k=>1] = -3.5;
B[j=>1,k=>2] = 0.2;
B[j=>1,k=>3] = 2.7;
B[j=>2,k=>3] = -1.0;
B[j=>3,k=>1] = -0.6;
B[j=>3,k=>3] = 4.2;
B[j=>4,k=>2] = -0.1;
B[j=>4,k=>3] = 1.7;

@show C = A*B;
```

```
C = A * B = ITensor ord=2
Dim 1: (dim=2|id=659)
Dim 2: (dim=3|id=647)
NDTensors.Dense{Float64, Vector{Float64}}
 2×3
```

```
 -7.0    0.21000000000000002  7.93
  0.06  -0.05                 3.43
```

[35]:
```
@assert isapprox(Array(C, i,k), [-7.0 0.21 7.93; 0.06 -0.05 3.43])
```

6. Array(): How to go from an ITensor to a Julia array?

Sometimes when you are testing your code, you need to compare the matrices with what you would do without ITensors (I do it all of the time). So, it is important to know how to move from an ITensor object to a standard array object in Julia. For this, we just need to use `Array()` and be careful about the indices.

Let's take as an example the tensor built in the previous exercise $C_{ik} = A_{ij} * B_{jk}$. Assuming that the first index of each tensor refers to the row and the second one to the columns, we would expect C_ik to be a 2×4 matrix.

[40]:
```
Array(C, i, k) #row, column
```

[40]:
```
2×3 Matrix{Float64}:
 -7.0    0.21  7.93
  0.06  -0.05  3.43
```

If you are not careful about this, you can get the transpose:

[41]:
```
Array(C, k, i)
```

[41]:
```
3×2 Matrix{Float64}:
 -7.0    0.06
  0.21  -0.05
  7.93   3.43
```

Clearly saying that one index is a column or a row is a convention; the point here is that **you need to be consistent**. Use the same convention that you are using in your standard linear algebra calculations.

This also works if you have tensors with more than one index.

[43]:
```
i = Index(2); # Index of dimension 2
j = Index(4); # Index of dimension 4
k = Index(3); # Index of dimension 3

A = ITensor(i,j,k);  # Define ITensor element by element

A[i=>1, j=>1, k=>3] = 2.0;
A[i=>1, j=>2, k=>3] = 0.7;
A[i=>1, j=>1, k=>3] = -3.5;
A[i=>1, j=>2, k=>3] = 0.2;

@show A;
```

```
A = ITensor ord=3
Dim 1: (dim=2|id=862)
Dim 2: (dim=4|id=748)
Dim 3: (dim=3|id=981)
NDTensors.Dense{Float64, Vector{Float64}}
 2×4×3
[:, :, 1] =
 0.0  0.0  0.0  0.0
 0.0  0.0  0.0  0.0

[:, :, 2] =
 0.0  0.0  0.0  0.0
 0.0  0.0  0.0  0.0

[:, :, 3] =
 -3.5  0.2  0.0  0.0
  0.0  0.0  0.0  0.0
```

6.1 Do you have a collection of 3 matrices of size 2x4, or a collection of 4 matrices of size 2x3?

[44]:
```
Array(A, i, j, k)
```

[44]:
```
2×4×3 Array{Float64, 3}:
[:, :, 1] =
 0.0  0.0  0.0  0.0
 0.0  0.0  0.0  0.0

[:, :, 2] =
 0.0  0.0  0.0  0.0
 0.0  0.0  0.0  0.0

[:, :, 3] =
 -3.5  0.2  0.0  0.0
  0.0  0.0  0.0  0.0
```

[45]:
```
Array(A, i, k, j)
```

[45]:
```
2×3×4 Array{Float64, 3}:
[:, :, 1] =
 0.0  0.0  -3.5
 0.0  0.0   0.0

[:, :, 2] =
 0.0  0.0  0.2
 0.0  0.0  0.0
```

```
[:, :, 3] =
 0.0  0.0  0.0
 0.0  0.0  0.0

[:, :, 4] =
 0.0  0.0  0.0
 0.0  0.0  0.0
```

Well, technically both options are correct. Only you know what your simulation is.

6.2 Exercise: Get a collection of 2 matrices with size 4 x 3

[46]:
```
Collection = Array(A, j, k, i)
```

[46]:
```
4×3×2 Array{Float64, 3}:
[:, :, 1] =
 0.0  0.0  -3.5
 0.0  0.0   0.2
 0.0  0.0   0.0
 0.0  0.0   0.0

[:, :, 2] =
 0.0  0.0  0.0
 0.0  0.0  0.0
 0.0  0.0  0.0
 0.0  0.0  0.0
```

[48]:
```
@assert Collection.size == (4,3,2)
```

7. Linear combinations of ITensors

ITensors may also be subtracted and multiplied by scalars, including complex scalars, for example:

[49]:
```
A = randomITensor(i,j,k)
B = randomITensor(k,i,j)
```

[49]:
```
ITensor ord=3 (dim=3|id=981) (dim=2|id=862) (dim=4|id=748)
NDTensors.Dense{Float64, Vector{Float64}}
```

[50]:
```
@show C = 4*A - B/2
@show D = A + 3.0im * B;
```

```
C = 4A - B / 2 = ITensor ord=3
Dim 1: (dim=2|id=862)
Dim 2: (dim=4|id=748)
Dim 3: (dim=3|id=981)
NDTensors.Dense{Float64, Vector{Float64}}
 2×4×3
```

```
[:, :, 1] =
  4.805207147441939   -0.8925523376789859  ...  -0.7520675724791618
 -1.9831017578688388   1.4515380846891315     -4.961236356207088

[:, :, 2] =
 1.6924441249072224  -0.459715179239932  ...  0.1374677751701282
 9.0661463515376     -5.352310454499417     3.6856263768253714

[:, :, 3] =
 -2.0249939156715833  -1.6214287527147624  ...  -0.8303898847233304
  0.8920686218311111  -3.786549461759092      -1.5008422776768915

D = A + (3.0im) * B = ITensor ord=3
Dim 1: (dim=2|id=862)
Dim 2: (dim=4|id=748)
Dim 3: (dim=3|id=981)
NDTensors.Dense{ComplexF64, Vector{ComplexF64}}
 2×4×3
[:, :, 1] =
   1.075415165086714 - 3.0212789225704952im  ...  -0.18919362928794195 -
0.028241668035636373im
 -0.6273122021325923 - 3.1568823039691827im       -1.10971323851292 +
3.13430041293245im

[:, :, 2] =
 0.17153244781123064 - 6.037886001973798im  ...  0.15942492995649998 +
3.0013916679352306im
  2.3276499942443274 + 1.466721752638251im      0.9877568087463499 +
1.5924051489601687im

[:, :, 3] =
 -0.5321582802419076 - 0.6218352317762823im  ...  -0.18006172043103635 +
0.6608580179951098im
   0.321449530101815 + 2.362376991456893im      -0.39235139887336307 -
0.41137990689936477im
```

This is just possible because A and B have the same indexes. Let's see what happens if this is not the case:

[51]:
```
l = Index(3)

A = randomITensor(i,j,k)
B = randomITensor(k,i,l)
```

[51]:
```
ITensor ord=3 (dim=3|id=981) (dim=2|id=862) (dim=3|id=631)
NDTensors.Dense{Float64, Vector{Float64}}
```

[52]:
```
C = 4*A - B/2
```

```
You are trying to add an ITensor with indices:

((dim=3|id=981), (dim=2|id=862), (dim=3|id=631))

into an ITensor with indices:

((dim=2|id=862), (dim=4|id=748), (dim=3|id=981))

but the indices are not permutations of each other.


Stacktrace:
 [1] error(s::String)
   @ Base .\error.jl:35
 [2] _map!!(f::Function, R::NDTensors.DenseTensor{...}, T1::NDTensors.
↪DenseTensor{...}, T2::NDTensors.DenseTensor{...})
   @ ITensors C:\Users\JOP204\.julia\packages\ITensors\a04AT\src\itensor.jl:1953
 [3] map!(f::Function, R::ITensor, T1::ITensor, T2::ITensor)
   @ ITensors C:\Users\JOP204\.julia\packages\ITensors\a04AT\src\itensor.jl:1983
 [4] copyto!
   @ C:\Users\JOP204\.julia\packages\ITensors\a04AT\src\broadcast.jl:336 [inlined
 [5] materialize!
   @ .\broadcast.jl:883 [inlined]
 [6] materialize!
   @ .\broadcast.jl:880 [inlined]
 [7] -(A::ITensor, B::ITensor)
   @ ITensors C:\Users\JOP204\.julia\packages\ITensors\a04AT\src\itensor.jl:1895
 [8] top-level scope
   @ In[52]:1
Some type information was truncated. Use `show(err)` to see complete types.
```

We already see that we can use * to contract the common indexes between two or more tensors. However, * can be used in different ways if we also use the functions prime(), delta(), combiner() and dag(). Let's go one by one.

8. prime(): level of a tensor's indices

Since indices are matched by identity (not just by dimension), sometimes we need two versions of "the same" index that ITensor treats as different — for example, when applying an operator to a state, the input and output physical indices must be distinguishable, even though they represent the same physical site with the same dimension. `prime()` solves this by adding a prime level to an index, effectively creating a labeled copy that has the same dimension and tags but a different identity for contraction purposes.

Consider two Tensors:

[53]:
```
A = randomITensor(i,j)
B = randomITensor(i,j);
```

8.1 How to get the common indices between different tensors?

[55]:
```
commoninds(A,B) #It returns the id of the common indixes between A and B
```

[55]:
```
2-element Vector{Index{Int64}}:
 (dim=2|id=862)
 (dim=4|id=748)
```

As we expected, these indixes are i and j:

[57]:
```
i,j
```

[57]:
```
((dim=2|id=862), (dim=4|id=748))
```

Then if we use * between A and B, the operator will contract both indexes:

[58]:
```
@show C = A*B; #We got a tensor of range zero (an scalar)
```

```
C = A * B = ITensor ord=0
NDTensors.Dense{Float64, Vector{Float64}}
 0-dimensional
-0.7597079126956108
```

This has sense because it is just $A_{i,j} * B_{i,j}$. IF we contract i and j, then we must have and scalar. We can access to this scalar in two ways:

[35]:
```
C[], scalar(C)
```

[35]:
```
(4.268102367055346, 4.268102367055346)
```

Some cases require us not to contract over all common indices.

8.2 Contracting only over one index even if there are two common indices

To avoid contraction over one common index (say i), we need to prime it in one of the tensors:

[59]:
```
A_prime = prime(A,i);
```

The ITensor A_prime has the same elements as A but has indices (i',j) instead of (i,j).

[61]:
```
@show A
@show A_prime;
```

```
A = ITensor ord=2
Dim 1: (dim=2|id=862)
Dim 2: (dim=4|id=748)
NDTensors.Dense{Float64, Vector{Float64}}
 2×4
```

```
 -0.5543623581602068   0.5953973173567123  ...  0.9126468051233392
  1.3213006005163777  -1.1721405163428074     0.7893246611124318

A_prime = ITensor ord=2
Dim 1: (dim=2|id=862)'
Dim 2: (dim=4|id=748)
NDTensors.Dense{Float64, Vector{Float64}}
 2×4
 -0.5543623581602068   0.5953973173567123  ...  0.9126468051233392
  1.3213006005163777  -1.1721405163428074     0.7893246611124318
```

You can also use `prime()` directly on an index object:

[63]:
```
prime(i), prime(prime(i))
```

[63]:
```
((dim=2|id=862)', (dim=2|id=862)'')
```

In case that you want to see how many primes you have you can use `plev()`:

[65]:
```
plev.(inds(A)) #Here I am using that . in Julia means to apply to all elements␣
 ↪of the list, and inds(A) gives a list with the indices of A
```

[65]:
```
(0, 0)
```

[67]:
```
plev.(inds(A_prime))
```

[67]:
```
(1, 0)
```

[69]:
```
plev(prime(i))
```

[69]:
```
1
```

You can also restart the prime level to zero by using `noprime()`:

[72]:
```
i'', noprime(i'')
```

[72]:
```
((dim=2|id=862)'', (dim=2|id=862))
```

We can also prime all indices of the tensor at the same time rather than just one:

[74]:
```
@show prime(A,i,j); #Same elem|ents as A but with indices (i',j') instead of␣
 ↪(i,j).
```

```
prime(A, i, j) = ITensor ord=2
Dim 1: (dim=2|id=862)'
Dim 2: (dim=4|id=748)'
NDTensors.Dense{Float64, Vector{Float64}}
 2×4
 -0.5543623581602068   0.5953973173567123  ...  0.9126468051233392
```

```
 1.3213006005163777  -1.1721405163428074     0.7893246611124318
```

[75]:
```
@show A'; #Another way to do it.
```

```
A' = ITensor ord=2
Dim 1: (dim=2|id=862)'
Dim 2: (dim=4|id=748)'
NDTensors.Dense{Float64, Vector{Float64}}
 2×4
 -0.5543623581602068   0.5953973173567123  ...  0.9126468051233392
  1.3213006005163777  -1.1721405163428074     0.7893246611124318
```

[75]:
```
ITensor ord=2 (dim=2|id=862)' (dim=4|id=748)'
NDTensors.Dense{Float64, Vector{Float64}}
```

In this case A_prime and B just have one common index:

[76]:
```
commoninds(A_prime,B)
```

[76]:
```
1-element Vector{Index{Int64}}:
 (dim=4|id=748)
```

So,

[77]:
```
@show C = A_prime*B;
```

```
C = A_prime * B = ITensor ord=2
Dim 1: (dim=2|id=862)'
Dim 2: (dim=2|id=862)
NDTensors.Dense{Float64, Vector{Float64}}
 2×2
 -2.2187830540206455  0.8195165727586886
 -3.220095917060091   1.4590751413250345
```

Instead of doing $A_{i,j} * B_{i,j}$, we did $A_{i',j} * B_{i,j} = C_{i',i}$.

Now if we want to get the same scalar as before, we just need to sum elements where $i = i'$. This is equivalent to multiplying by the Kronecker delta tensor.

9. delta(): Identity or Kronecker delta tensor

This function creates a Kronecker delta tensor: a tensor that is 1 when all its indices take the same value, and 0 otherwise. It's the tensor-network equivalent of an identity map.

[80]:
```
@show delta(i,i');
```

```
delta(i, i') = ITensor ord=2
Dim 1: (dim=2|id=862)
```

```
Dim 2: (dim=2|id=862)'
NDTensors.Diag{Float64, Float64}
 2×2
 1.0  0.0
 0.0  1.0
```

Then,

[81]:
```
@show C*delta(i,i');
```

```
C * delta(i, i') = ITensor ord=0
NDTensors.Dense{Float64, Vector{Float64}}
 0-dimensional
-0.759707912695611
```

In general, delta() output is the identity tensor, we can build it with more indexes:

[82]:
```
@show delta(i,j,k);
```

```
delta(i, j, k) = ITensor ord=3
Dim 1: (dim=2|id=862)
Dim 2: (dim=4|id=748)
Dim 3: (dim=3|id=981)
NDTensors.Diag{Float64, Float64}
 2×4×3
[:, :, 1] =
 1.0  0.0  0.0  0.0
 0.0  0.0  0.0  0.0

[:, :, 2] =
 0.0  0.0  0.0  0.0
 0.0  1.0  0.0  0.0

[:, :, 3] =
 0.0  0.0  0.0  0.0
 0.0  0.0  0.0  0.0
```

Indeed, Julia allows us to use the symbol δ directly instead of delta.

[83]:
```
@show δ(k,i,j) == delta(i,j,k);
```

```
δ(k, i, j) == delta(i, j, k) = true
```

9.1 How to replace indices?

Sometimes we want to change one index for another one. Let's say that you define two matrices with different indices, but you want to multiply them. For this, they need a common index. We

can exchange indices with the identity, simply noting that $A_j = \sum_i B_i \delta_{i,j}$. Usually, thi|s is done considering the same dimension.

[85]:
```
i = Index(3); # Index of dimension 2
j = Index(3); # Index of dimension 2
@show i,j

A = randomITensor(i)
@show A;

A = A*delta(i,j)
@show A;
```

```
(i, j) = ((dim=3|id=869), (dim=3|id=660))
A = ITensor ord=1
Dim 1: (dim=3|id=869)
NDTensors.Dense{Float64, Vector{Float64}}
 3-element
  0.7018060520873951
 -0.22076453514736927
  0.25865652083340707

A = ITensor ord=1
Dim 1: (dim=3|id=660)
NDTensors.Dense{Float64, Vector{Float64}}
 3-element
  0.7018060520873951
 -0.22076453514736927
  0.25865652083340707
```

9.2 Tracing out some degrees of freedom.

We could think that maybe we can split the legs of the tensor using deltas, but it is dangerous. Consider you have A_i and you want to reshape it as a matrix A_{kl}.

[88]:
```
i = Index(2); # Index of dimension 2
j = Index(2); # Index of dimension 2
m = Index(4)
@show A = randomITensor(m);
```

```
A = randomITensor(m) = ITensor ord=1
Dim 1: (dim=4|id=411)
NDTensors.Dense{Float64, Vector{Float64}}
 4-element
 -0.15600690583347995
 -0.8258880490127793
  0.46098432171651965
 -0.533437830161736
```

You could multiply with a `delta(m,j,k)` and it will become a matrix, but **it will not necessarily have all the information**. The right process for reshaping the matrix should use the `combiner()` function explain in the next section.

[89]:
```
@show A*delta(m,j,k);
```

```
A * delta(m, j, k) = ITensor ord=2
Dim 1: (dim=2|id=972)
Dim 2: (dim=3|id=981)
NDTensors.Dense{Float64, Vector{Float64}}
 2×3
 -0.15600690583347995   0.0                  0.0
  0.0                  -0.8258880490127793   0.0
```

This property of "losing" information is very useful to trace out freedom degrees (a very common operation in quantum mechanics). One example of performing the partial trace in quantum mechanics can be found here: https://itensor.discourse.group/t/trace-and-partial-trace-of-mpo/79/2

In conclusion, `delta()` can be very useful but you need to be careful about how you want to use it.

[90]:
```
A = randomITensor(i,j,l); #Suppose that we want to trace out j, l and just keep␣
 ↪i.
@show A*delta(j,l); # Very useful but also dangerous.
```

```
A * delta(j, l) = ITensor ord=1
Dim 1: (dim=2|id=229)
NDTensors.Dense{Float64, Vector{Float64}}
 2-element
 -0.2096960470891009
 -0.006224522926706499
```

10. Exercise: Contract over $_i$ in $A_{ij}B_{ji}$ and get a rank-1 tensor.

We want $C_j = \sum_i A_{ij}B_{ji}$. From implementing the definition this would be:

[91]:
```
i = Index(3); # Index of dimension 2
j = Index(4); # Index of dimension 4

A = randomITensor(i,j)
B = randomITensor(j,i)
```

[91]:
```
ITensor ord=2 (dim=4|id=147) (dim=3|id=387)
NDTensors.Dense{Float64, Vector{Float64}}
```

[93]:
```
C_from_definition = ITensor(j)
```

```
for j_index = 1:dim(j)
    C_from_definition[j => j_index] = 0.0
    for i_index = 1:dim(i)
        C_from_definition[j => j_index] += A[i => i_index, j => j_index]*B[j =>␣
 ↪j_index, i => i_index]
    end
end

@show C_from_definition;
```

```
C_from_definition = ITensor ord=1
Dim 1: (dim=4|id=147)
NDTensors.Dense{Float64, Vector{Float64}}
 4-element
 -3.7476761273492745
  0.733797163317693
 -0.4262969105872601
  1.0375432134848654
```

However, the idea of the exercise is to use `prime()` and `delta()` to get the same tensor.

- **Hint 1:** You need to use primes to avoid contraction over j.
- **Hint 2:** $A_{ij}B_{j'i}$ gives a tensor with indices j, j' and C has only one index.
- **Hint 3:** You need to define a new index by analyzing $j' = j$. You can do this multiplying by $\delta_{j,j',j''}$.

[96]:
```
B_prime = prime(B, j)
C = A*B_prime  #We have two indices here instead of one, like in the definition.␣
 ↪What should we do?
@show inds(C);

C = C*delta(j, j', j'') # We need to sum over j, j', but instead of building a␣
 ↪scalar, we need to define a new index (this can be j'')
C = noprime(C);

@show C;
```

```
inds(C) = ((dim=4|id=147), (dim=4|id=147)')
C = ITensor ord=1
Dim 1: (dim=4|id=147)
NDTensors.Dense{Float64, Vector{Float64}}
 4-element
 -3.7476761273492745
  0.7337971633176931
 -0.42629691058726005
  1.0375432134848654
```

[97]:
```
@assert isapprox(C, C_from_definition)
```

11. Exercise: Contract over $_{i,j}$ in $A_{ij}B_{jk}C_{km}D_{mi}$ and get a rank-2 tensor.

We want $F_{km} = \sum_{i,j} A_{ij}B_{jk}C_{km}D_{mi}$

[101]:
```
i = Index(3); # Index of dimension 2
j = Index(4); # Index of dimension 4
k = Index(3); # Index of dimension 2
m = Index(5); # Index of dimension 4

A = randomITensor(i,j)
B = randomITensor(j,k)
C = randomITensor(k,m)
D = randomITensor(m,i);
```

[102]:
```
F_from_definition = ITensor(k, m)

for k_index = 1:dim(k)
    for m_index = 1:dim(m)
        F_from_definition[k => k_index, m => m_index] = 0.0
        for i_index = 1:dim(i)
            for j_index = 1:dim(j)
                F_from_definition[k => k_index, m => m_index] += A[i => i_index,␣
 ↪j => j_index]*B[j => j_index, k => k_index]*C[k => k_index, m => m_index]*D[m␣
 ↪=> m_index, i => i_index]
            end
        end
    end
end
```

[103]:
```
B_prime = prime(B, k)
D_prime = prime(D, m)

F = A * B_prime * C * D_prime
@show inds(F); #We have four indices here instead of 2, like in the definition.␣
 ↪What should we do?
F = F * delta(k, k', k'') * delta(m, m', m'') # We need to sum over k,k' and␣
 ↪m,m' and define a new set of indices.
F = noprime(F); #Let's go to k and m
@show F;
```

```
inds(F) = ((dim=3|id=565)', (dim=3|id=565), (dim=5|id=395), (dim=5|id=395)')
F = ITensor ord=2
Dim 1: (dim=3|id=565)
Dim 2: (dim=5|id=395)
```

```
NDTensors.Dense{Float64, Vector{Float64}}
 3×5
  7.725641989175154    0.5967195766303645  ...   1.1279966523681124
 -0.29376420799561487  1.6258410401206698      0.3470804387897324
  0.2248882753958451   2.622994841698467       0.4597455024274289
```

[104]:
```
@assert isapprox(F, F_from_definition); #No sure if it was simpler to be honest␣
↪haha.
```

Maybe right now `prime()` doesn't look very useful, but actually it is. For example, it can be used to create density matrix operators from quantum states without contracting all the indices in the process.

12. `combiner()`: How to mix tensor legs?

Sometimes we need to merge several indices into a single one (or split one index back into several) — for example, turning a rank-3 tensor A_{ijk} into a matrix $A_{(ij),k}$ before doing an SVD.

[107]:
```
i = Index(3); # Index of dimension 2
j = Index(4); # Index of dimension 4
k = Index(3); # Index of dimension 2
l = Index(5); # Index of dimension 4

A = randomITensor(i,j,k,l);
```

We have a tensor of order 4. If we want to reshape it as a tensor of order 2, we can use the function combiner to merge i, j and k indices:

[110]:
```
@show Mix_i_j_k = combiner(i,j,k, tags = "ijk"); #tags is an optional parameter␣
↪when we define indexes, is just a label to know what means the index. In this␣
↪case is useful to remember which indexes was merged.
```

```
Mix_i_j_k = combiner(i, j, k, tags = "ijk") = ITensor ord=4
Dim 1: (dim=36|id=669|"ijk")
Dim 2: (dim=3|id=378)
Dim 3: (dim=4|id=745)
Dim 4: (dim=3|id=48)
NDTensors.Combiner
 36×3×4×3

Permutation of blocks: Int64[]
Combination of blocks: Int64[]
```

Then we just need to do the product:

[111]:
```
@show A_reshaped_ijk_l = Mix_i_j_k*A;
```

```
A_reshaped_ijk_l = Mix_i_j_k * A = ITensor ord=2
Dim 1: (dim=36|id=669|"ijk")
Dim 2: (dim=5|id=889)
NDTensors.Dense{Float64, Vector{Float64}}
 36×5
  0.956756072717872     -0.191540845145784    ...   0.5119698259828743
 -0.4356471929284932     1.3236758032067393       0.6910062374484539
 -1.112004413953904     -0.712311468812024       -0.7963002311687071
 -0.9644678924683378    -0.15948677643403858     -0.7407302801234785
 -2.105197783988303     -0.5835091187557373       0.27817856750922826
 -1.8390045650572788    -0.5429801599761115   ...  -0.13414857587949822
  2.0479371596542495     0.9636331011788597      -0.593745746847357
  1.040326748764619      1.9207397721160524       0.9036547267410914
  0.6868044225852032     0.8391249051588113      -2.0977611285614457
 -0.5528141425503038     1.517303327671139        0.3918313490591644
  2.628959786224317      0.24701988431332592  ...  -0.8216006586162691
 -0.2492208146809082    -0.2889073405296461      -0.002501427384770408
 -0.7331085782285301    -1.211325755719447        1.881169317535535

 -1.79541371336533      -0.6670409695726603      -1.7636525441622994
 -0.36089536778756987    1.5558811640717651   ...  -1.2280446487012577
  0.3202524058629947     1.3276006977773787       0.6842172685601822
  1.0871711187099051     0.5345829863265238       1.3706716847857139
 -0.4669493481408341    -0.22659609616185442     -0.15790944681578534
  0.9334367365444234     0.4176841443867246       1.6190280128456553
  1.4336666625122771     1.4331927073354636   ...   1.3113399926631517
 -0.42383228743128704    0.14062076738415666     -0.8840088817046832
  0.4816385098397814     0.678491394992567       -0.40055332626991874
  0.038407044985620344   0.5932350203295866       0.09365094297775291
 -1.3021587987829404    -0.865045041672648       -1.2607149913562334
  0.7724089275172482    -0.3653436238303149   ...  -0.6650663178464489
```

12.1 Exercise: Combine i, j and k, l to have a matrix of dimensions dim(i)*dim(j)* x *dim(k)*dim(l).

For this, use the same A as before.

Hint: You need to define two combiners.

[116]:
```
Mix_i_j = combiner(i,j, tags = "ij")
Mix_k_l = combiner(k,l, tags = "kl")

@show A_reshaped_ij_kl = Mix_i_j*(Mix_k_l*A)
```

```
A_reshaped_ij_kl = Mix_i_j * (Mix_k_l * A) = ITensor ord=2
Dim 1: (dim=12|id=486|"ij")
Dim 2: (dim=15|id=257|"kl")
NDTensors.Dense{Float64, Vector{Float64}}
```

```
12×15
 0.956756072717872   -0.7331085782285301   ...  -1.7636525441622994
-0.4356471929284932   0.6248625437862513       -1.2280446487012577
-1.112004413953904    1.3414448475254726        0.6842172685601822
-0.9644678924683378   0.0720181821638612        1.3706716847857139
-2.105197783988303   -0.7492327311378526       -0.15790944681578534
-1.8390045650572788  -1.7832469477665145   ...   1.6190280128456553
 2.0479371596542495  -0.4012538792669054        1.3113399926631517
 1.040326748764619   -1.0768491350942464       -0.8840088817046832
 0.6868044225852032   0.49687155516730974      -0.40055332626991874
-0.5528141425503038   0.5835770424693527        0.09365094297775291
 2.628959786224317    1.727982470717676    ...  -1.2607149913562334
-0.2492208146809082  -1.2464561742031832       -0.6650663178464489
```

[116]:
```
ITensor ord=2 (dim=12|id=486|"ij") (dim=15|id=257|"kl")
NDTensors.Dense{Float64, Vector{Float64}}
```

[117]:
```
@assert dims(A_reshaped_ij_kl) == (dim(i)*dim(j), dim(k)*dim(l))
```

What about reshaping a 4-dimensional index back into two 2-dimensional ones? This is a reshape, and unlike `delta()` (which only picks out a diagonal and loses information), `combiner()` can be used to perform a genuine, information-preserving reshape. However, in that scenario `combiner()` is not the whole story, we need another function.

13. `dag()`: How to split tensor legs? & How to get adjoint operators?

This function (short for "dagger") returns the complex conjugate of an ITensor's elements.

What does this have to do with splitting legs and reshaping? A combiner can be viewed mathematically as a rectangular isometric matrix C, i.e., combining indices (mix the legs) behaves like multiplying by C. Therefore, to undo that multiplication (split the legs), you must multiply by its inverse or adjoint, $C^\dagger$.

To recover the initial shape we jused need to do the following:

[121]:
```
A_reconstructed_1 = dag(Mix_i_j_k)*A_reshaped_ijk_l
A_reconstructed_2 = dag(Mix_k_l)*(dag(Mix_i_j)*A_reshaped_ij_kl)

# Let's see if these tensor are the same as A:
@show A == A_reconstructed_1 == A_reconstructed_2;
```

```
A == A_reconstructed_1 == A_reconstructed_2 = true
```

With this in mind, let's go back to the example where `delta()` did not work to reshape the matrices:

[124]:
```
m = Index(4)
@show A = randomITensor(m);
```

```
A = randomITensor(m) = ITensor ord=1
Dim 1: (dim=4|id=30)
NDTensors.Dense{Float64, Vector{Float64}}
 4-element
 -0.8674375924824366
  1.4171892733284612
  0.30360975496002357
  0.26775403226628214
```

[125]:
```
i = Index(2); # Index of dimension 2
j = Index(2); # Index of dimension 2
@show A*delta(m,j,k);
```

```
A * delta(m, j, k) = ITensor ord=2
Dim 1: (dim=2|id=61)
Dim 2: (dim=3|id=48)
NDTensors.Dense{Float64, Vector{Float64}}
 2×3
 -0.8674375924824366  0.0                  0.0
  0.0                 1.4171892733284612  0.0
```

[127]:
```
C = combiner(i, j) # combiner: (k,l) <-> one combined index of dim 4
ci = combinedind(C) # this is the dim-4 combined index.
A = A*delta(m, ci) #We need to change the index of our matrix to the combined␣
 ↪index
@show A*dag(C) #Now we apply the dagger operation to split the combined index.
```

```
A * dag(C) = ITensor ord=2
Dim 1: (dim=2|id=764)
Dim 2: (dim=2|id=61)
NDTensors.Dense{Float64, Vector{Float64}}
 2×2
 -0.8674375924824366  0.30360975496002357
  1.4171892733284612  0.26775403226628214
```

[127]:
```
ITensor ord=2 (dim=2|id=764) (dim=2|id=61)
NDTensors.Dense{Float64, Vector{Float64}}
```

Now, let's start considering complex numbers to see that this function is actually performing the dagger.

[129]:
```
@show A = randomITensor(i) + 1im*randomITensor(i) # This creates a complex␣
 ↪tensor; at the end is a linear combination using a complex scalar in the sum.␣
 ↪1im = i in Julia.
```

```
A = randomITensor(i) + (1im) * randomITensor(i) = ITensor ord=1
Dim 1: (dim=2|id=764)
```

```
NDTensors.Dense{ComplexF64, Vector{ComplexF64}}
 2-element
 0.18102157806438435 + 0.9548340874704784im
  1.8361428201832883 - 1.0058411056395755im
```

[129]:
```
ITensor ord=1 (dim=2|id=764)
NDTensors.Dense{ComplexF64, Vector{ComplexF64}}
```

[131]:
```
@show dag(A)
```

```
dag(A) = ITensor ord=1
Dim 1: (dim=2|id=764)
NDTensors.Dense{ComplexF64, Vector{ComplexF64}}
 2-element
 0.18102157806438435 - 0.9548340874704784im
  1.8361428201832883 + 1.0058411056395755im
```

[131]:
```
ITensor ord=1 (dim=2|id=764)
NDTensors.Dense{ComplexF64, Vector{ComplexF64}}
```

Remember that **tensors do not have a notion of inputs and outputs; they only contain indices that can be contracted with other tensors**. Remember that `dag()` applies the conjugate to all tensors; we still need to decide which index is the input and which is the output of the operator.

Consider a 2x3 matrix.

[133]:
```
A_matrix = [1.0 2.0im 3.0 + 2im; 4.0im 5.0 6.0im]

j = Index(3) #input
i = Index(2) #output
A = ITensor(A_matrix,i,j)
@show A;
```

```
A = ITensor ord=2
Dim 1: (dim=2|id=124)
Dim 2: (dim=3|id=583)
NDTensors.Dense{ComplexF64, Vector{ComplexF64}}
 2×3
 1.0 + 0.0im  0.0 + 2.0im  3.0 + 2.0im
 0.0 + 4.0im  5.0 + 0.0im  0.0 + 6.0im
```

[135]:
```
@assert Matrix(A, i,j) == Array(A, i,j); #This is another way to go from tensors␣
 ↪to standard julia objects.
Matrix(A, i,j)
```

```
[135]: 2×3 Matrix{ComplexF64}:
        1.0+0.0im  0.0+2.0im  3.0+2.0im
        0.0+4.0im  5.0+0.0im  0.0+6.0im
```

```
[137]: A[i=>1,j=>2], A_matrix[1, 2]
```

```
[137]: (0.0 + 2.0im, 0.0 + 2.0im)
```

```
[138]: A_matrix_dag = adjoint(A_matrix)
```

```
[138]: 3×2 adjoint(::Matrix{ComplexF64}) with eltype ComplexF64:
        1.0-0.0im  0.0-4.0im
        0.0-2.0im  5.0-0.0im
        3.0-2.0im  0.0-6.0im
```

```
[139]: Adag = dag(A)
```

```
[139]: ITensor ord=2 (dim=2|id=124) (dim=3|id=583)
       NDTensors.Dense{ComplexF64, Vector{ComplexF64}}
```

```
[140]: Matrix(Adag, i, j) #Always the first index that you put on Matrix will␣
        ↪correspond to columns and the second to rows. You can think of each row as a␣
        ↪collection of columns.
```

```
[140]: 2×3 Matrix{ComplexF64}:
        1.0-0.0im  0.0-2.0im  3.0-2.0im
        0.0-4.0im  5.0-0.0im  0.0-6.0im
```

```
[141]: Matrix(Adag, j, i)
```

```
[141]: 3×2 Matrix{ComplexF64}:
        1.0-0.0im  0.0-4.0im
        0.0-2.0im  5.0-0.0im
        3.0-2.0im  0.0-6.0im
```

```
[142]: Adag[i=>2,j=>3], A_matrix_dag[3,2]
```

```
[142]: (0.0 - 6.0im, 0.0 - 6.0im)
```

Let's see why is important to keep this in mind. Sometimes in the research you would need to implement in the tensor networks language an operation that in math notation looks like $A^\dagger A$ (I do it all the time for open quantum systems). This means that the output of A should be contracted with the input of $A^\dagger$.

```
[143]: adjoint(A_matrix)*A_matrix
```

```
[143]: 3×3 Matrix{ComplexF64}:
        17.0+0.0im    0.0-18.0im  27.0+2.0im
         0.0+18.0im  29.0+0.0im    4.0+24.0im
```

```
 27.0-2.0im    4.0-24.0im  49.0+0.0im
```

[144]:
```
Adag = dag(A)
Adag = Adag*delta(j, j') #We just want to contract the output of A (i) with the input of Adag (i). We need to rename the output of Adag in order to avoid that contraction.
Matrix(A*Adag, j, j')
```

[144]:
```
3×3 Matrix{ComplexF64}:
 17.0+0.0im    0.0+18.0im  27.0-2.0im
  0.0-18.0im  29.0+0.0im    4.0-24.0im
 27.0+2.0im    4.0+24.0im  49.0+0.0im
```

13.1 Exercise: Implement $AA^\dagger$

[145]:
```
Adag = dag(A)
Adag = Adag*delta(i, i')
```

[145]:
```
ITensor ord=2 (dim=3|id=583) (dim=2|id=124)'
NDTensors.Dense{ComplexF64, Vector{ComplexF64}}
```

[146]:
```
@assert isapprox(A_matrix*adjoint(A_matrix), Matrix(A*Adag, i, i'))
```

In conclusion, when you use `dag()` think about what you really want to contract.

14. `qr()` and `svd()`: Descomposition of ITensors

QR and SVD are two of the most useful and well-known ways to decompose a matrix (rank-2 tensor):

$$M = QR, \qquad M = USV^\dagger$$

where Q is orthogonal ($Q^\dagger Q = \mathbb{1}$) and R is upper-triangular for QR, while U, V are orthogonal/unitary and S is diagonal with non-negative singular values for SVD.

In principle, if we have a higher-rank tensor with indices i, j, k, l, decomposing would consists on the following steps:

- Reshape it into a matrix by grouping indices into a row group and a column group.
- Apply the matrix decomposition
- Reshape the result back so that the output tensors carry the correct original indices.

Fortunately, **qr()** and **svd()** do all of this internally for us.

All we need to do is tell the function which indices belong to the "left'' side (i.e. which indices should end up on Q, or on U) — every other index is automatically grouped on the other side.

[150]:
```
i = Index(3); # Index of dimension 2
j = Index(4); # Index of dimension 4
k = Index(3); # Index of dimension 2
l = Index(5); # Index of dimension 4

A = randomITensor(i,j,k,l);
```

[151]:
```
Q,R = qr(A, (i,j)); #Q will have the indices (i,j)
@show Q;
```

```
Q = ITensor ord=3
Dim 1: (dim=3|id=715)
Dim 2: (dim=4|id=670)
Dim 3: (dim=12|id=782|"Link,qr")
NDTensors.Dense{Float64, Vector{Float64}}
 3×4×12
[:, :, 1] =
 -0.4335648868760422   -0.20395686183401798  ...  -0.03978700969908887
 -0.5957774285500815   -0.055809989134192         0.4862273661404238
 -0.01820949739803169  -0.1826956107203268        0.3542071172453711

[:, :, 2] =
 -0.2406548025553913    -0.06272860506379009   ...   0.19393354186923928
 -0.17254266134003537   -0.013586979384572859     -0.10201078399705159
 -0.007638801877647045   0.45971912443549195       0.024946244216478577

[:, :, 3] =
 -0.3283708382143984   -0.2002699519524962     ...  -0.6401378178163185
  0.47999435508280786   0.0038344758038901636      0.05161426118393175
 -0.34913514268134227  -0.019182102695066514       0.13391515437640023

;;; ...

[:, :, 10] =
  0.3752864836246144   0.1011354030298694     ...  -0.32480301041105064
 -0.5330201519918958   0.30727191614853894       -0.35421704961097267
 -0.44180458362606456  0.0036406103872991258      0.08081136197277328

[:, :, 11] =
 -0.0930670345056843    0.09308251344963468  ...  -0.009622048478726275
  0.039237365698855255  0.04979169776557686      0.09166362380485538
 -0.058245838001302445  0.41879201057069115      0.037516249610664534

[:, :, 12] =
 0.24122969094326155  -0.37881689362712645  ...  -0.000721824904611923
 0.11341022753613479   0.13572238836905787     -0.2471937380335243
 0.4519525226328955   -0.2771096604283357       0.5561383433696658
```

14.1 Exercise: Check that ITensors does all the mixing/splitting for us for `qr()`

[152]:
```
Mix_i_j = combiner(i,j, tags = "ij")
Mix_k_l = combiner(k,l, tags = "kl")
A_reshaped_ij_kl = Mix_i_j*(Mix_k_l*A)

@show inds(A_reshaped_ij_kl)
@show ind(A_reshaped_ij_kl, 1);

Q_longprocess, R_longprocess = qr(A_reshaped_ij_kl, ind(A_reshaped_ij_kl, 1));␣
 ↪#Q will have the index ind(A_reshaped_ij_kl, 1)
QR_common_index = commonind(Q_longprocess, R_longprocess)

Q_longprocess = Q_longprocess*dag(Mix_i_j)*delta(QR_common_index, commonind(Q,␣
 ↪R))
R_longprocess = R_longprocess*dag(Mix_k_l)*delta(QR_common_index, commonind(Q,␣
 ↪R))

@assert isapprox(Q_longprocess, Q)
@assert isapprox(R_longprocess, R)
```

```
inds(A_reshaped_ij_kl) = ((dim=12|id=19|"ij"), (dim=15|id=58|"kl"))
ind(A_reshaped_ij_kl, 1) = (dim=12|id=19|"ij")
```

Similary for `svd()`

[153]:
```
U,S,V = svd(A, (i,j)); #U will have the indices (i,j)
@show U
@show S
@show V
```

```
U = ITensor ord=3
Dim 1: (dim=3|id=715)
Dim 2: (dim=4|id=670)
Dim 3: (dim=12|id=620|"Link,u")
NDTensors.Dense{Float64, Vector{Float64}}
 3×4×12
[:, :, 1] =
  0.456083915026922    -0.27377835628990543  ...    0.023596781385066518
  0.06425920806555185   0.07676196676890032      -0.30086986431264195
 -0.07149427260610043  -0.24687502551994778      -0.4774686591725116

[:, :, 2] =
  0.07626325518866582   0.40641120149547194  ...    0.4586863985325097
  0.4647501968367586    0.1195922488678012       -0.4815295417267177
 -0.1482374087726365   -0.15693001881441515       0.05567408697820447

[:, :, 3] =
  0.27505051377152245  0.3872887751041224    ...    0.30337395869122485
```

```
 -0.21221149563060532  0.10169554045371985      0.18294466571746706
  0.6450468987302866   0.23713140545203104     -0.3396197161907542

;;; ...

[:, :, 10] =
  0.35370552389556154  0.34358278949880294  ...  -0.4827824144951848
 -0.3749290164302249   0.1644394571031526       -0.14554747760076908
 -0.3795719671410894   0.16897338900296205      -0.14489075348975505

[:, :, 11] =
  0.2042599904757705   -0.2198705929574273   ...  -0.16921215993157127
 -0.18373938598361292   0.19791834824742516     -0.14477295252176314
  0.449533428968089    -0.6138017629854192       0.2954746381636818

[:, :, 12] =
  0.1277429494578109    -0.1186970642396524   ...  -0.12174573659666912
 -0.022646380599325694   0.08691706885697018     -0.2938737685539497
  0.1385297730415666     0.2860140143942337       0.3373271652265203

S = ITensor ord=2
Dim 1: (dim=12|id=620|"Link,u")
Dim 2: (dim=12|id=923|"Link,v")
NDTensors.Diag{Float64, Vector{Float64}}
 12×12
 6.231981010190247  0.0                ...  0.0
 0.0                6.024625961842563     0.0
 0.0                0.0                   0.0
 0.0                0.0                   0.0
 0.0                0.0                   0.0
 0.0                0.0                ...  0.0
 0.0                0.0                   0.0
 0.0                0.0                   0.0
 0.0                0.0                   0.0
 0.0                0.0                   0.0
 0.0                0.0                ...  0.0
 0.0                0.0                   0.5869754158830124

V = ITensor ord=3
Dim 1: (dim=3|id=387)
Dim 2: (dim=5|id=837)
Dim 3: (dim=12|id=923|"Link,v")
NDTensors.Dense{Float64, Vector{Float64}}
 3×5×12
[:, :, 1] =
 0.1582135864543025    0.2544074537678047  ...  -0.3735587526457943
 0.388530834258546     0.0892325752976709      0.05444657261753788
 0.11140730805031766  -0.3717894829867125      0.03681184119046564
```

```
[:, :, 2] =
 0.26211492907985817  0.24935276642726567  ...   0.06196582991869796
 0.06580152269880743  0.20282128467608895      0.18618542585754028
 0.1475882468620407   0.14316743012222555     -0.5111915540313886

[:, :, 3] =
  0.09616519567119275   -0.31204550376669504  ...  -0.14976948574166438
 -0.017146077160654555  -0.22656028157529381     -0.058163777193872473
  0.4791841146503669    -0.13968665272730096      0.027675450298271898

;;; ...

[:, :, 10] =
  0.24587564388575192   0.17722840795717446  ...   0.2648025037442021
 -0.16961138544716048  -0.06189177695863143     -0.0978032446413184
  0.04695149010864783   0.31568639437922785      0.510332812757019

[:, :, 11] =
 -0.4180432269408962   -0.2270088902235548   ...   0.18719671519691997
 -0.08820353334323613   0.12452501947128225      0.4177652109357381
  0.10768231169560077  -0.10627637017809427     -0.05160012702597662

[:, :, 12] =
  0.33492578077284935  -0.30049410431270057   ...   0.1947403464426353
 -0.3153812773091903    0.024354870687622307      0.2501497742618468
 -0.14137990357138364  -0.416892184262883        -0.24878097910459648
```

[153]:
```
ITensor ord=3 (dim=3|id=387) (dim=5|id=837) (dim=12|id=923|"Link,v")
NDTensors.Dense{Float64, Vector{Float64}}
```

Let's practice a little bit more

[155]:
```
i = Index(3, "i")
j = Index(3, "j")
k = Index(3, "k")

A = randomITensor(i,j,k);
```

[156]:
```
Q,R = qr(A,(i,j))

@show inds(Q) #Has i,j
@show inds(R); #Has k
```

```
inds(Q) = ((dim=3|id=296|"i"), (dim=3|id=932|"j"), (dim=3|id=911|"Link,qr"))
inds(R) = ((dim=3|id=911|"Link,qr"), (dim=3|id=524|"k"))
```

or

[160]:
```
Q,R = qr(A,(i))

@show inds(Q) #Has i
@show inds(R); #Has j,k
```

```
inds(Q) = ((dim=3|id=296|"i"), (dim=3|id=687|"Link,qr"))
inds(R) = ((dim=3|id=687|"Link,qr"), (dim=3|id=932|"j"), (dim=3|id=524|"k"))
```

In any case,

[161]:
```
A  Q*R
```

[161]:
```
true
```

With SVD descomposition is exactly the same:

[162]:
```
U,S,V = svd(A,(i,j))

@show inds(U) # Has i,j
@show inds(S)
@show inds(V); # Has k
```

```
inds(U) = ((dim=3|id=296|"i"), (dim=3|id=932|"j"), (dim=3|id=857|"Link,u"))
inds(S) = ((dim=3|id=857|"Link,u"), (dim=3|id=929|"Link,v"))
inds(V) = ((dim=3|id=524|"k"), (dim=3|id=929|"Link,v"))
```

[163]:
```
U,S,V = svd(A,(i))

@show inds(U) # Has i
@show inds(S)
@show inds(V); # Has j,k
```

```
inds(U) = ((dim=3|id=296|"i"), (dim=3|id=293|"Link,u"))
inds(S) = ((dim=3|id=293|"Link,u"), (dim=3|id=870|"Link,v"))
inds(V) = ((dim=3|id=932|"j"), (dim=3|id=524|"k"), (dim=3|id=870|"Link,v"))
```

In any case,

[164]:
```
 U*S*V  A # true
```

[164]:
```
true
```

14.2 What about the reduced SVD and the truncation error?

[165]:
```
i = Index(10)
j = Index(40)
k = Index(20)

A = randomITensor(i,j,k) #Is very very big.
```

[165]:
```
ITensor ord=3 (dim=10|id=161) (dim=40|id=691) (dim=20|id=790)
NDTensors.Dense{Float64, Vector{Float64}}
```

[168]:
```
U, S, V = svd(A, (i,k));
@show error = (norm(U*S*V - A)/norm(A))^2; #it contains all the singular values
singular_values = diag(Matrix(S, inds(S)));
```

```
error = (norm(U * S * V - A) / norm(A)) ^ 2 = 5.84660272319366e-30
```

When performing an SVD, we often don't want to keep all singular values — especially for large systems, where the bond dimension can grow exponentially. maxdim and cutoff are the two main knobs used to control this truncation.

14.3 Maxdim - the maximum number of singular values (bond dimension) to keep

[169]:
```
U, S, V = svd(A, (i,k), maxdim=5); #We are truncating and getting a reduced SVD␣
↪with just 5 singular values
@show S
@show error = (norm(U*S*V - A)/norm(A))^2;
```

```
S = ITensor ord=2
Dim 1: (dim=5|id=408|"Link,u")
Dim 2: (dim=5|id=219|"Link,v")
NDTensors.Diag{Float64, Vector{Float64}}
 5×5
 19.498559902469893   0.0                 ...   0.0
  0.0                19.298747645021727       0.0
  0.0                 0.0                     0.0
  0.0                 0.0                     0.0
  0.0                 0.0                    17.954509114503068

error = (norm(U * S * V - A) / norm(A)) ^ 2 = 0.7777676799197805
```

[170]:
```
U, S, V = svd(A, (i,k), maxdim=10); #We are truncating and getting a reduced SVD␣
↪with just 10 singular values
@show S
@show error = (norm(U*S*V - A)/norm(A))^2;
```

```
S = ITensor ord=2
Dim 1: (dim=10|id=527|"Link,u")
Dim 2: (dim=10|id=197|"Link,v")
NDTensors.Diag{Float64, Vector{Float64}}
 10×10
 19.498559902469893   0.0                 ...   0.0
  0.0                19.298747645021727       0.0
  0.0                 0.0                     0.0
  0.0                 0.0                     0.0
  0.0                 0.0                     0.0
```

```
 0.0                 0.0                 ...   0.0
 0.0                 0.0                     0.0
 0.0                 0.0                     0.0
 0.0                 0.0                     0.0
 0.0                 0.0                   16.17245364493514

error = (norm(U * S * V - A) / norm(A)) ^ 2 = 0.5987602697540291
```

14.4 Cutoff - discards singular values whose relative contribution to the total weight falls below this threshold

[172]:
```
U, S, V = svd(A, (i,k), cutoff = 0.1); #It will take the necessary singular␣
 ↪values to have an error of 0.1
@show inds(S) #the number of singular values is the dimension of the index.
@show error = (norm(U*S*V - A)/norm(A))^2;
```

```
inds(S) = ((dim=32|id=564|"Link,u"), (dim=32|id=521|"Link,v"))
error = (norm(U * S * V - A) / norm(A)) ^ 2 = 0.09119366897050421
```

14.5 Exercise: Calculate the truncation error

Check that the last `(norm(U*S_10*V - A)/norm(A))^2` printed is equivalent to calculate $\frac{\sum_{i=\chi+1}^{N}\lambda_i^2}{\sum_{i=1}^{N}\lambda_i^2}$ with the corresponding number of singular values. For this, you can use the `singular_values` variable defined from the exact SVD.

[173]:
```
χ = dim(S,1)
sum(singular_values[χ+1:end].^2)/sum(singular_values[1:end].^2)
```

[173]:
```
0.09119366897050417
```

This reveals the square root of the Frobenius norm is ignored by ITensors. This is the reference value that ITensors uses for truncation when you use `cutoff`.

14.6 What if we have maxdim and cutoff at the same time?

When we have both `maxdim` and `cutoff`, the decomposition will the necessary singular values to have an error of `cutoff` and then take just take a subset of `maxdim` singular values. Clearly, in this case the error could be bigger than `cutoff`.

[174]:
```
U, S, V = svd(A, (i,k), maxdim = 5, cutoff = 0.1); #It will take the necessary␣
 ↪singular values to have an error of 0.1 and then take just 5 singular values,␣
 ↪the error could be bigger than 0.1.
@show inds(S) #the number of singular values is the dimension of the index.
@show error = (norm(U*S*V - A)/norm(A))^2;
```

```
inds(S) = ((dim=5|id=142|"Link,u"), (dim=5|id=714|"Link,v"))
error = (norm(U * S * V - A) / norm(A)) ^ 2 = 0.7777676799197805
```

Note: Most of tensor networks require using `svd()` internally, so most of the functions receive also `maxdim` and `cutoff` as parameters (more on this later).

15. Matrix Product States (MPS)

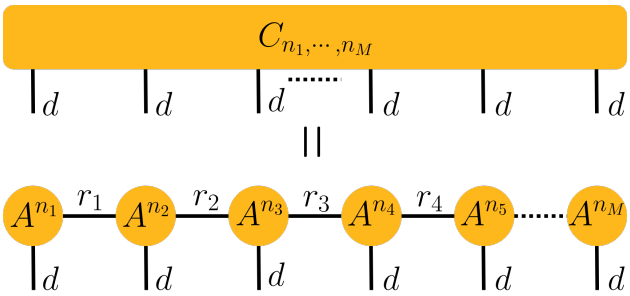
Now that we know how to decompose tensors with SVD and control truncation, we're ready to tackle the real motivation behind all of this: representing many-body quantum states efficiently. A generic quantum state of M sites, each with local dimension d, is described by a rank-M tensor with d^M coefficients — a number that grows exponentially and quickly becomes intractable, even for modest M. A **Matrix Product State (MPS)** rewrites this single, unwieldy tensor as a **chain of smaller tensors**, one per site, connected by the bond indices we saw appear naturally out of SVD:

$$|\psi\rangle = \sum_{n_1,\cdots,n_M=1}^{d} C_{n_1,\cdots,n_M} |n_1,\cdots,n_M\rangle$$

$$C_{n_1,\ldots,n_M} = \sum_{a_1=1}^{r_1} \sum_{a_2=1}^{r_2} \sum_{a_3=1}^{r_3} \cdots \sum_{a_{M-1}}^{r_{M-1}} A^{n_1}_{a_1} A^{\sigma_2}_{a_1,a_2} A^{\sigma_3}_{a_2,a_3} \ldots A^{\sigma_{M-1}}_{a_{M-2},a_{M-1}} A^{n_M}_{a_{M-1}}$$

Instead of storing d^M numbers, we now only need $O(Md\chi^2)$, where χ is the bond dimension — the very quantity we learned to control via `maxdim` and `cutoff`. In this section, we'll construct MPS explicitly — by hand and by decomposing a full state vector via repeated SVDs — and compare against exact diagonalization (ED) for small systems, so you can see directly how the MPS approximation behaves as we vary the bond dimension and truncation error.

In diagram notation:



The first step in defining an MPS is creating a set of indices for the legs.

[175]:
```
i = Index(2)
j = Index(2)
k = Index(2)
l = Index(2)
m = Index(2)

C = randomITensor(i,j,k,l,m) # From ITensor
```

[175]:
```
ITensor ord=5 (dim=2|id=227) (dim=2|id=392) (dim=2|id=363) (dim=2|id=948)
(dim=2|id=974)
```

```
NDTensors.Dense{Float64, Vector{Float64}}
```

[176]:
```
cutoff = 1E-20
maxdim = 10
ψ = MPS(C,(i,j,k,l,m) ;cutoff=cutoff,maxdim=maxdim)
```

[176]:
```
5-element MPS:
 ((dim=2|id=227), (dim=2|id=919|"Link,l=1"))
 ((dim=2|id=919|"Link,l=1"), (dim=2|id=392), (dim=4|id=417|"Link,l=2"))
 ((dim=4|id=417|"Link,l=2"), (dim=2|id=363), (dim=4|id=800|"Link,l=3"))
 ((dim=4|id=800|"Link,l=3"), (dim=2|id=948), (dim=2|id=799|"Link,l=4"))
 ((dim=2|id=799|"Link,l=4"), (dim=2|id=974))
```

[179]:
```
@show linkdims(ψ); #List with all bond dimension.
@show maxlinkdim(ψ); #Maximum bond dimension.
```

```
linkdims(ψ) = [2, 4, 4, 2]
maxlinkdim(ψ) = 4
```

If we want to recover C we just need to contract all tensors inside ψ. We could do it by multiplying one by one with `*`, or we could simply use `contract()`

[180]:
```
@assert isapprox(contract(ψ), C)
```

Just for pedagogical purposes, in principle, we could also build the MPS doing the SVDs by ourselves and storing the tensors at the end.

[181]:
```
# --- Step 1: split off site i ---
U1, S1, V1 = svd(C, (i,); cutoff=cutoff, maxdim=maxdim) # U1 has indices (i,␣
 ↪bond1) -> this is our first MPS tensor, A1
A1 = U1
bond1 = commonind(U1, S1)
R1 = S1 * V1    # # absorb S1 into the remainder so we can keep decomposing.␣
 ↪indices: (bond1, j, k, l, m)

# --- Step 2: split off site j ---
U2, S2, V2 = svd(R1, (bond1, j); cutoff=cutoff, maxdim=maxdim)
A2 = U2                      # indices: (bond1, j, bond2)
bond2 = commonind(U2, S2)
R2 = S2 * V2                 # indices: (bond2, k, l, m)

# --- Step 3: split off site k ---
U3, S3, V3 = svd(R2, (bond2, k); cutoff=cutoff, maxdim=maxdim)
A3 = U3                      # indices: (bond2, k, bond3)
bond3 = commonind(U3, S3)
R3 = S3 * V3                 # indices: (bond3, l, m)

# --- Step 4: split off site l ---
U4, S4, V4 = svd(R3, (bond3, l); cutoff=cutoff, maxdim=maxdim)
```

```
A4 = U4                       # indices: (bond3, l, bond4)
bond4 = commonind(U4, S4)
R4 = S4 * V4                  # indices: (bond4, m)

# --- Step 5: last site, nothing left to split ---
A5 = R4                       # indices: (bond4, m)
```

[181]:
```
ITensor ord=2 (dim=2|id=867|"Link,u") (dim=2|id=974)
NDTensors.Dense{Float64, Vector{Float64}}
```

[182]:
```
ψ_from_definition = MPS(ITensor[A1,A2,A3,A4,A5])
```

[182]:
```
5-element MPS:
 ((dim=2|id=227), (dim=2|id=930|"Link,u"))
 ((dim=2|id=930|"Link,u"), (dim=2|id=392), (dim=4|id=571|"Link,u"))
 ((dim=4|id=571|"Link,u"), (dim=2|id=363), (dim=4|id=555|"Link,u"))
 ((dim=4|id=555|"Link,u"), (dim=2|id=948), (dim=2|id=867|"Link,u"))
 ((dim=2|id=867|"Link,u"), (dim=2|id=974))
```

[183]:
```
norm(ψ_from_definition - ψ)
```

[183]:
```
8.126508339566158e-16
```

To be honest, I have **NEVER** done this long process, haha. It is better to just use `MPS()`, but I thought that it was worth seeing once in your life during the tutorial.

15.1 `siteinds()`: "SiteTypes" to represent different physical degrees of freedom provided by ITensors.

When building an MPS "by hand" as we just did, every index was created manually and had no notion of what kind of physical system it represented — it was just an abstract dimension-2 leg. In practice, this becomes a real burden as soon as you want to apply operators, since you'd need to build every operator matrix (Pauli matrices, raising/lowering operators, number operators…) yourself and keep track of index conventions consistently across the whole chain.

The `siteinds()` function solves this by creating indices that are aware of the local physical Hilbert space they represent — for example, siteinds("S=1/2", N) creates N spin-1/2 indices, while siteinds("Boson", N; dim=…) creates bosonic indices with a chosen truncation.

[187]:
```
N = 10; # Number of sites
sites = siteinds("Qubit", N) # Indices for all sites
```

[187]:
```
10-element Vector{Index{Int64}}:
 (dim=2|id=957|"Qubit,Site,n=1")
 (dim=2|id=652|"Qubit,Site,n=2")
 (dim=2|id=317|"Qubit,Site,n=3")
 (dim=2|id=918|"Qubit,Site,n=4")
 (dim=2|id=985|"Qubit,Site,n=5")
 (dim=2|id=636|"Qubit,Site,n=6")
```

```
 (dim=2|id=871|"Qubit,Site,n=7")
 (dim=2|id=702|"Qubit,Site,n=8")
 (dim=2|id=281|"Qubit,Site,n=9")
 (dim=2|id=587|"Qubit,Site,n=10")
```

This means that ITensors is considering the next local basis for building states and operators:

$$|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \quad , \quad |1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

Let's see another example of `SiteType`.

[188]:
```
siteinds("Electron", N) # Indices for all sites
```

[188]:
```
10-element Vector{Index{Int64}}:
 (dim=4|id=963|"Electron,Site,n=1")
 (dim=4|id=277|"Electron,Site,n=2")
 (dim=4|id=483|"Electron,Site,n=3")
 (dim=4|id=784|"Electron,Site,n=4")
 (dim=4|id=461|"Electron,Site,n=5")
 (dim=4|id=879|"Electron,Site,n=6")
 (dim=4|id=724|"Electron,Site,n=7")
 (dim=4|id=277|"Electron,Site,n=8")
 (dim=4|id=19|"Electron,Site,n=9")
 (dim=4|id=763|"Electron,Site,n=10")
```

Now the dimension of each index is 4 instead of 2. This is because ITensors is considering a local basis

$$|0\rangle = \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \end{pmatrix}, \quad |\uparrow\rangle = \begin{pmatrix} 0 \\ 1 \\ 0 \\ 0 \end{pmatrix}, \quad |\downarrow\rangle = \begin{pmatrix} 0 \\ 0 \\ 1 \\ 0 \end{pmatrix}, \quad |\uparrow\downarrow\rangle = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \end{pmatrix}$$

However, let's focus on the qubit scenario.

15.2 Defining states from ket notation.

Let's say that we want to build the state $|0000000000\rangle$ for a qubit system with 10 sites.

[189]:
```
ket_state = ["0" for n=1:N];
ψ = MPS(sites, ket_state)
```

[189]:
```
10-element MPS:
 ((dim=2|id=957|"Qubit,Site,n=1"), (dim=1|id=773|"Link,l=1"))
 ((dim=1|id=773|"Link,l=1"), (dim=2|id=652|"Qubit,Site,n=2"),
(dim=1|id=627|"Link,l=2"))
 ((dim=1|id=627|"Link,l=2"), (dim=2|id=317|"Qubit,Site,n=3"),
(dim=1|id=91|"Link,l=3"))
 ((dim=1|id=91|"Link,l=3"), (dim=2|id=918|"Qubit,Site,n=4"),
```

```
(dim=1|id=48|"Link,l=4"))
 ((dim=1|id=48|"Link,l=4"), (dim=2|id=985|"Qubit,Site,n=5"),
(dim=1|id=41|"Link,l=5"))
 ((dim=1|id=41|"Link,l=5"), (dim=2|id=636|"Qubit,Site,n=6"),
(dim=1|id=161|"Link,l=6"))
 ((dim=1|id=161|"Link,l=6"), (dim=2|id=871|"Qubit,Site,n=7"),
(dim=1|id=512|"Link,l=7"))
 ((dim=1|id=512|"Link,l=7"), (dim=2|id=702|"Qubit,Site,n=8"),
(dim=1|id=490|"Link,l=8"))
 ((dim=1|id=490|"Link,l=8"), (dim=2|id=281|"Qubit,Site,n=9"),
(dim=1|id=220|"Link,l=9"))
 ((dim=1|id=220|"Link,l=9"), (dim=2|id=587|"Qubit,Site,n=10"))
```

15.3 `expect()`: Calculating expectation values.

To check that this is the right state we can calculate $\langle Z_i \rangle$ for each site. We would expect $\langle Z_i \rangle = 1 \forall i$.

[190]:
```
@show expect(ψ, "Z");
```

```
expect(ψ, "Z") = [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
```

15.4 Comparison with exact diagonalization objects

In principle, we could also build the exact state from just standard linear algebra, and load it to ITensors.

[194]:
```
zero = [1; 0]
one = [0; 1]

ψ_ED = zero
for i = 1:N-1
    ψ_ED = kron(zero, ψ_ED)
end

@show ψ_ED;
```

```
ψ_ED = [1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
```

```
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0]
```

[195]:
```
function Build_Z_j(N::Int64, j::Int64) #Builds Z_j as I...IZI...I for a system of N sites.
    Z = [1 0; 0 -1]
    Identity = [1 0; 0 1]

    M = Identity
    j == 1 ? M = Z : nothing
    for i=2:N
        i == j ? M = kron(Z, M) :  M = kron(Identity, M) #It is more efficient to save everything as sparse matrices.
    end

    return M
end

Z_i = [adjoint(ψ_ED)*Build_Z_j(N,i)*ψ_ED for i = 1:N];
@show Z_i;
```

```
Z_i = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
```

Ok, so it is the same object. Let's try to load it to ITensors.

[196]:
```
ψ_fromED = MPS(ψ_ED, sites, cutoff = 1e-15)
```

[196]:
```
10-element MPS:
 ((dim=2|id=957|"Qubit,Site,n=1"), (dim=1|id=698|"Link,l=1"))
 ((dim=1|id=698|"Link,l=1"), (dim=2|id=652|"Qubit,Site,n=2"),
(dim=1|id=825|"Link,l=2"))
 ((dim=1|id=825|"Link,l=2"), (dim=2|id=317|"Qubit,Site,n=3"),
(dim=1|id=909|"Link,l=3"))
 ((dim=1|id=909|"Link,l=3"), (dim=2|id=918|"Qubit,Site,n=4"),
(dim=1|id=165|"Link,l=4"))
 ((dim=1|id=165|"Link,l=4"), (dim=2|id=985|"Qubit,Site,n=5"),
(dim=1|id=688|"Link,l=5"))
 ((dim=1|id=688|"Link,l=5"), (dim=2|id=636|"Qubit,Site,n=6"),
(dim=1|id=254|"Link,l=6"))
 ((dim=1|id=254|"Link,l=6"), (dim=2|id=871|"Qubit,Site,n=7"),
(dim=1|id=812|"Link,l=7"))
 ((dim=1|id=812|"Link,l=7"), (dim=2|id=702|"Qubit,Site,n=8"),
(dim=1|id=762|"Link,l=8"))
 ((dim=1|id=762|"Link,l=8"), (dim=2|id=281|"Qubit,Site,n=9"),
(dim=1|id=195|"Link,l=9"))
 ((dim=1|id=195|"Link,l=9"), (dim=2|id=587|"Qubit,Site,n=10"))
```

[198]:
```
@show expect(ψ_fromED, "Z");
```

```
expect(ψ_fromED, "Z") = [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
```

Ok, everything is consistent. We could also contract our initial MPS implementation and compare with the ED vector.

[199]:
```
ψ_contracted = contract(ψ)
```

[199]:
```
ITensor ord=10 (dim=2|id=957|"Qubit,Site,n=1") (dim=2|id=652|"Qubit,Site,n=2")
(dim=2|id=317|"Qubit,Site,n=3") (dim=2|id=918|"Qubit,Site,n=4")
(dim=2|id=985|"Qubit,Site,n=5") (dim=2|id=636|"Qubit,Site,n=6")
(dim=2|id=871|"Qubit,Site,n=7") (dim=2|id=702|"Qubit,Site,n=8")
(dim=2|id=281|"Qubit,Site,n=9") (dim=2|id=587|"Qubit,Site,n=10")
NDTensors.Dense{Float64, Vector{Float64}}
```

[200]:
```
comb = combiner(sites) #Our ED object has only one index; we need to combine all␣
 ↪sites into just 1.
@assert Array(ψ_contracted*comb, combinedind(comb)) == ψ_ED
```

15.5 Exercise: Define $|0,0,0,0,0,1,1,1,1,1\rangle$ in MPS, check $\langle Z_i \rangle$ and compare with the ED object.

[205]:
```
# ED case: Try both cases, but only the first one gives the same expectation␣
 ↪values, why?
ψ_ED = kron(one, one, one, one, one, zero, zero, zero, zero, zero);
```

```
# ψ_ED = kron(zero, zero, zero, zero, zero, one, one, one, one, one);

Z_i = [adjoint(ψ_ED)*Build_Z_j(N,i)*ψ_ED for i = 1:N];
@show Z_i;
```

```
Z_i = [1, 1, 1, 1, 1, -1, -1, -1, -1, -1]
```

[207]:
```
ket_state = ["0", "0", "0", "0", "0", "1", "1", "1", "1", "1"];
ψ = MPS(sites, ket_state)
@show expect(ψ, "Z");

ψ_contracted = contract(ψ)
comb = combiner(sites) #Our ED object has only one index; we need to combine all␣
 ↪sites into just 1.
@assert Array(ψ_contracted*comb, combinedind(comb)) == ψ_ED
```

```
expect(ψ, "Z") = [1.0, 1.0, 1.0, 1.0, 1.0, -1.0, -1.0, -1.0, -1.0, -1.0]
```

This reveals that ITensors, internally, when building the states, is thinking of multiplying the tensor products to the left, rather than to the right as you would do by hand.

15.6 Exercise: Define $|0,1,0,1,0,1,0,1,0,1\rangle$ in MPS, check $\langle Z_i \rangle$ and compare with the ED object.

[123]:
```
ψ_ED = kron(one, zero, one, zero, one, zero, one, zero, one, zero);

ket_state = ["0", "1", "0", "1", "0", "1", "0", "1", "0", "1"];
ψ = MPS(sites, ket_state)
@show expect(ψ, "Z");

ψ_contracted = contract(ψ)
comb = combiner(sites) #Our ED object has only one index; we need to combine all␣
 ↪sites into just 1.
@assert Array(ψ_contracted*comb, combinedind(comb)) == ψ_ED
```

```
expect(ψ, "Z") = [1.0, -1.0, 1.0, -1.0, 1.0, -1.0, 1.0, -1.0, 1.0, -1.0]
```

15.7 `randomMPS()` - 6:30PM

This function construct a real, random Matrix Product State (MPS) with a specified link (bond) dimension.

[208]:
```
ψ1 = randomMPS(sites,linkdims=10);
ψ2 = randomMPS(sites,linkdims=10);
ψ3 = randomMPS(sites,linkdims=10)
```

[208]:
```
10-element MPS:
 ((dim=2|id=957|"Qubit,Site,n=1"), (dim=10|id=371|"Link,l=1"))
 ((dim=10|id=371|"Link,l=1"), (dim=2|id=652|"Qubit,Site,n=2"),
```

```
(dim=10|id=711|"Link,l=2"))
 ((dim=10|id=711|"Link,l=2"), (dim=2|id=317|"Qubit,Site,n=3"),
(dim=10|id=732|"Link,l=3"))
 ((dim=10|id=732|"Link,l=3"), (dim=2|id=918|"Qubit,Site,n=4"),
(dim=10|id=432|"Link,l=4"))
 ((dim=10|id=432|"Link,l=4"), (dim=2|id=985|"Qubit,Site,n=5"),
(dim=10|id=707|"Link,l=5"))
 ((dim=10|id=707|"Link,l=5"), (dim=2|id=636|"Qubit,Site,n=6"),
(dim=10|id=56|"Link,l=6"))
 ((dim=10|id=56|"Link,l=6"), (dim=2|id=871|"Qubit,Site,n=7"),
(dim=8|id=771|"Link,l=7"))
 ((dim=8|id=771|"Link,l=7"), (dim=2|id=702|"Qubit,Site,n=8"),
(dim=4|id=211|"Link,l=8"))
 ((dim=4|id=211|"Link,l=8"), (dim=2|id=281|"Qubit,Site,n=9"),
(dim=2|id=384|"Link,l=9"))
 ((dim=2|id=384|"Link,l=9"), (dim=2|id=587|"Qubit,Site,n=10"))
```

15.8 Addition of MPSs

A natural operation we'd like to do is add two MPS together, e.g. $|\psi\rangle = |\phi_1\rangle + |\phi_2\rangle$ (this comes up constantly — building superpositions, applying a sum of operators, Trotterized time evolution, etc.). But there's an important catch: the bond dimension of the sum is (at most) the sum of the bond dimensions of the two inputs, not their maximum.

[210]:
```
ψ = ψ2 + ψ3 # A cutoff of 1e-15 is used by default
@show linkdims(ψ);
```

```
linkdims(ψ) = [4, 8, 16, 20, 20, 16, 8, 4, 2]
```

[211]:
```
ψ = +(ψ2, ψ3; cutoff = 1e-8); # Set the cutoff
@show linkdims(ψ);
```

```
linkdims(ψ) = [4, 8, 16, 20, 20, 16, 8, 4, 2]
```

[212]:
```
ψ = +(ψ2, ψ3; maxdim = 10); # Set maxdim
@show linkdims(ψ);
```

```
linkdims(ψ) = [4, 8, 10, 10, 10, 10, 8, 4, 2]
```

15.9 norm()

This function computes the norm of a tensor or MPS, i.e., $\sqrt{\langle\psi|\psi\rangle}$.

[213]:
```
@show norm(ψ);
```

```
norm(ψ) = 1.229472615421661
```

[214]:
```
ψ_normalized = ψ/norm(ψ)
@show norm(ψ_normalized);
```

```
ψ_normalized = normalize(ψ)
@show norm(ψ_normalized);

normalize!(ψ) #A lot of functions receive the ! and perform an in-place␣
↪operation, be careful with this.
@show norm(ψ);
```

```
norm(ψ_normalized) = 1.0
norm(ψ_normalized) = 1.0
norm(ψ) = 1.0
```

15.10 `truncate()`

To truncate an MPS, you do not need to do the `svd()` yourself; you can just call this function.

[215]:
```
ψ_truncated = truncate(ψ, cutoff = 1e-1)
@show linkdims(ψ_truncated);
```

```
linkdims(ψ_truncated) = [2, 3, 4, 5, 6, 7, 6, 4, 2]
```

[216]:
```
ψ_truncated = truncate(ψ, maxdim = 1)
@show linkdims(ψ_truncated);
```

```
linkdims(ψ_truncated) = [1, 1, 1, 1, 1, 1, 1, 1, 1]
```

15.11 Exercise: What is the bond dimension used for a cutoff of 0.02?

[220]:
```
ψ_truncated = truncate(ψ, cutoff = 1.0)
@show maxlinkdim(ψ_truncated);
```

```
maxlinkdim(ψ_truncated) = 1
```

15.12 `inner()`, `op()` and `apply()`: Expectation values and overlaps of MPS

Two of the most common quantities we compute with MPS are overlaps between states, $\langle\phi|\psi\rangle$, and expectation values of operators, $\langle\psi|\hat{O}|\psi\rangle$ — both of which are handled efficiently without ever constructing the full state vector. `inner(φ, ψ)` computes the overlap $\langle\phi|\psi\rangle$ by contracting the two MPS site-by-site (automatically applying `dag()` to φ internally), while `op("OpName", sites[n])` builds a local operator (e.g. "Sz", "Sx", "N") acting on the physical site index `s[n]`, ready to be applied to the MPS with `apply()`. Combining the three lets us compute expectation values cleanly: apply the operator to a copy of the state, then take the overlap with the original.

[223]:
```
inner(ψ_truncated,ψ) #Check what happens if you change cutoff or maxdim in the␣
↪previous lines.
```

[223]:
```
0.033442305719853016
```

[224]:
```
dot(ψ_truncated,ψ) #This gives the same result. It exists mainly for consistency␣
↪with Julia's LinearAlgebra. However, I prefer inner because it is also␣
↪compatible with products like <Φ|H|Φ> if H is an MPO.
```

[224]:
```
0.033442305719853016
```

Now, let's define an apply some operators.

Assume we start from $|0,1,0,1,0,1,0,1,0,1\rangle$

[225]:
```
ket_state = ["0", "1", "0", "1", "0", "1", "0", "1", "0", "1"];
ψ = MPS(sites, ket_state);
```

[227]:
```
@show op("Z", sites[1]);
```

```
op("Z", sites[1]) = ITensor ord=2
Dim 1: (dim=2|id=957|"Qubit,Site,n=1")'
Dim 2: (dim=2|id=957|"Qubit,Site,n=1")
NDTensors.Dense{Float64, Vector{Float64}}
 2×2
 1.0   0.0
 0.0  -1.0
```

[229]:
```
Z1 = op("Z", sites[1])
commoninds(ψ[1], Z1) #common indices are inputs and prime indices are outputs.
```

[229]:
```
1-element Vector{Index{Int64}}:
 (dim=2|id=957|"Qubit,Site,n=1")
```

To act the operator we need to use `apply()`

[231]:
```
ψ_afterZ1 = apply(Z1, ψ; cutoff = 1e-8, maxdim = 4) #This also can receive␣
↪maxdim and cutoff as aprameters.
```

[231]:
```
10-element MPS:
 ((dim=2|id=957|"Qubit,Site,n=1"), (dim=1|id=429|"Link,l=1"))
 ((dim=2|id=652|"Qubit,Site,n=2"), (dim=1|id=371|"Link,l=2"),
(dim=1|id=429|"Link,l=1"))
 ((dim=2|id=317|"Qubit,Site,n=3"), (dim=1|id=569|"Link,l=3"),
(dim=1|id=371|"Link,l=2"))
 ((dim=2|id=918|"Qubit,Site,n=4"), (dim=1|id=859|"Link,l=4"),
(dim=1|id=569|"Link,l=3"))
 ((dim=2|id=985|"Qubit,Site,n=5"), (dim=1|id=210|"Link,l=5"),
(dim=1|id=859|"Link,l=4"))
 ((dim=2|id=636|"Qubit,Site,n=6"), (dim=1|id=747|"Link,l=6"),
(dim=1|id=210|"Link,l=5"))
 ((dim=2|id=871|"Qubit,Site,n=7"), (dim=1|id=0|"Link,l=7"),
(dim=1|id=747|"Link,l=6"))
 ((dim=2|id=702|"Qubit,Site,n=8"), (dim=1|id=103|"Link,l=8"),
(dim=1|id=0|"Link,l=7"))
 ((dim=2|id=281|"Qubit,Site,n=9"), (dim=1|id=857|"Link,l=9"),
(dim=1|id=103|"Link,l=8"))
```

```
((dim=2|id=587|"Qubit,Site,n=10"), (dim=1|id=857|"Link,l=9"))
```

[139]:
```
@show inner(ψ, ψ_afterZ1)
@show inner(ψ, apply(Z1, ψ));

Zi = [inner(ψ, apply(op("Z", sites[i]), ψ)) for i = 1:N];
@show Zi;
```

```
inner(ψ, ψ_afterZ1) = 1.0
inner(ψ, apply(Z1, ψ)) = 1.0
Zi = [1.0, -1.0, 1.0, -1.0, 1.0, -1.0, 1.0, -1.0, 1.0, -1.0]
```

We could think on applying Hadarmards gates and then check $\langle X_i \rangle$. Remember that \$H $|0\rangle = |+\rangle$\$and$H|1\rangle = |-\rangle$.

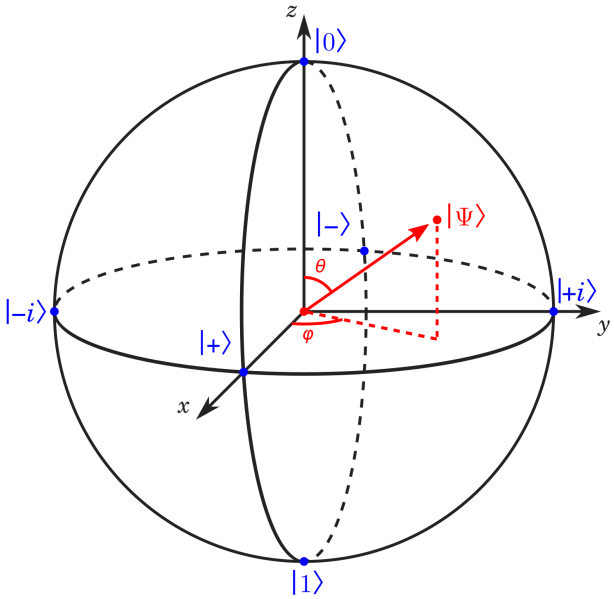
[233]:
```
for i = 1:N
    ψ = apply(op("H", sites[i]), ψ)
end

Xi = [inner(ψ, apply(op("X", sites[i]), ψ)) for i = 1:N];
@show Xi;
```

```
Xi = [-4.474228634151464e-17, 0.0, 4.7231600470208783e-17, 0.0,
2.818318481941687e-17, 0.0, 9.13476916862494e-18, 0.0, -9.913646482167006e-18,
3.607329692369476e-17]
```

15.13 Exercise: Define $|001100\rangle$, apply a rotation of $\theta = \pi/2$ at all sites and calculate $\langle Y_i \rangle$

Remember the Bloch sphere:



Hint: You need to define the operator "Rx" which also takes ϑ as an argument (the rotation angle around the x-axis).

[141]:
```
N = 6; # Number of sites
sites = siteinds("Qubit", N) # Indices for all sites
ket_state = ["0", "0", "1", "1", "0", "0"];
ψ = MPS(sites, ket_state);

for i = 1:N
    Rx_gate = op("Rx", sites[i]; θ=π/2)
    ψ = apply(Rx_gate, ψ)
end

Yi = [inner(ψ, apply(op("Y", sites[i]), ψ)) for i = 1:N];
@show Yi;
```

```
Yi = ComplexF64[-0.9999999999999989 + 0.0im, -0.9999999999999989 + 0.0im,
0.9999999999999991 + 0.0im, 0.9999999999999991 + 0.0im, -0.9999999999999991 +
0.0im, -0.9999999999999992 + 0.0im]
```

[142]:
```
@assert isapprox(Yi, [-1, -1, 1, 1, -1, -1])
```

15.14 von-Neumann entanglement entropy

We've already seen that the SVD singular values S_α carry physical meaning — they control how much of the state's structure survives truncation at a given bond. The von Neumann entropy makes this connection precise: for a bipartition of the chain into a left and right half at bond n, the entanglement entropy between the two halves is computed directly from the singular values at that cut,

$$S_n = -\sum_\alpha p_\alpha \log p_\alpha, \qquad p_\alpha = S_\alpha^2$$

where p_α are the (normalized) squared singular values, i.e. the eigenvalues of the reduced density matrix on either side of the cut. This is one of the most important diagnostic quantities in tensor network methods: it tells us exactly how entangled the state is across that bond, which in turn tells us how large the bond dimension χ needs to be to represent the state accurately.

[238]:
```
using Plots
using LaTeXStrings

function Calculate_SvN_entropy(state)

    N = length(siteinds(state))
    SvN = zeros(N)
    for b=1:N
        orthogonalize!(state, b)
        if b == 1
```

```
            U,S,V = svd(state[b], (siteind(state, b))) # There is no link to the␣
 ↪left
        else
            U,S,V = svd(state[b], (linkind(state, b-1), siteind(state, b)))
        end
        for n=1:dim(S, 1)
            p = S[n,n]^2
            SvN[b] -= p * log(p)
        end
    end

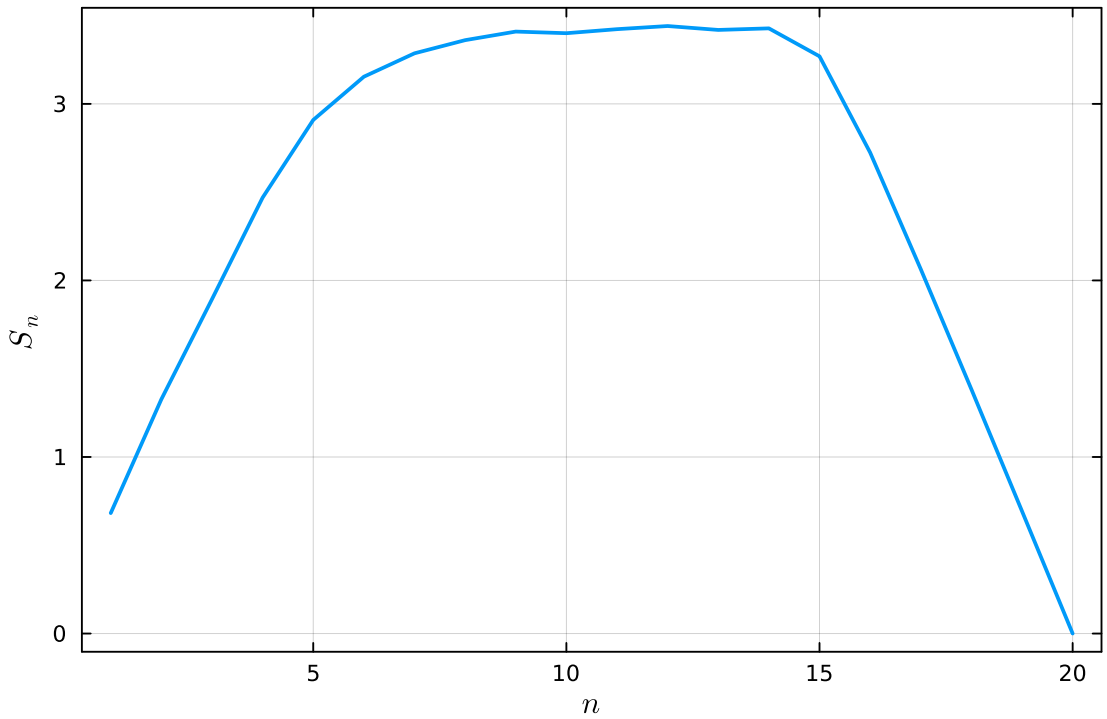
    return SvN
end
```

[238]:
```
Calculate_SvN_entropy (generic function with 1 method)
```

[144]:
```
N = 20; # Number of sites
sites = siteinds("Qubit", N) # Indices for all sites
ψ = randomMPS(sites,linkdims=40);

SvN = Calculate_SvN_entropy(ψ)
plot(1:20, SvN, xlabel = L"n", ylabel = L"S_{n}", frame = :box, dpi = 200,␣
 ↪linewidth = 2.0, label = false)
```

[144]:



15.15 Encoding functions and Hadamard product (element-wise multiplication)

So far, every operation we've discussed — applying operators, computing overlaps, adding states — treats the MPS as a genuine quantum state living in a linear vector space. But there's an entirely different, equally important operation: the Hadamard product, i.e., multiplying two MPS element-wise, index by index, rather than contracting them as an inner product or superposing them as a sum:

$$(\psi \odot \phi)_{s_1 s_2 \cdots s_N} = \psi_{s_1 s_2 \cdots s_N} \cdot \phi_{s_1 s_2 \cdots s_N}$$

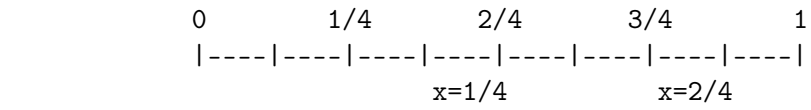
This is not a standard quantum-mechanical operation — there's no physical "state" interpretation of multiplying two amplitudes together — but it becomes essential the moment we start using MPS to represent functions rather than quantum states. If ψ encodes a function $f(x)$ and ϕ encodes $g(x)$ (both discretized/binarized across the same N physical indices), then $\psi \odot \phi$ encodes the pointwise product $f(x)g(x)$ — directly enabling us to compute non-linear combinations of functions already stored in MPS form, such as $f(x)^2$, without ever leaving the compressed tensor-network representation.

Before explaining how to implement the Hadamard product, **let's discuss how to encode a function with qubits**.

Consider $f(x)$ a function with $x \in [0, 1]$. The domain can be discretized through N qubits into 2^N grid points $x_n = n/2^N$, for $n = 0, 1, \ldots, 2^N - 1$.

Example with $N = 4$:

$$x_n = \{0,\ 1/16,\ 2/16,\ \ldots,\ 15/16\}$$



Note that we can encode x_n with 4 qubits, i.e. for x_n there exist $n_1, n_2, n_3, n_4 \in \{0, 1\}$ such that:

$$x_n = x_{n_1 n_2 n_3 n_4} = \frac{n_1}{2} + \frac{n_2}{2^2} + \frac{n_3}{2^3} + \frac{n_4}{2^4}, \qquad n_j = 0, 1$$

$$\left(x_{12} = x_{1100} = \frac{1}{2} + \frac{1}{4} = \frac{3}{4} \right)$$

Binary encoding table (each column is one value $n = 0, \ldots, 15$; rows are the qubit values $n_1 n_2 n_3 n_4$ — column $n = 12$, i.e. 1100, is highlighted):

n	0	1	2	3	4	5	6	7	8	9	10	11	12	**13**	14	15
n_1	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1
n_2	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
n_3	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
n_4	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1

We can do the same with a function!

$$f(x) \longrightarrow \{f(x_n)\}_{n=1,\ldots,N}$$

$$|\psi\rangle = \sum_{n_1,\ldots,n_N=0,1} f_{n_1 n_2 \ldots n_N} |n_1\rangle \otimes |n_2\rangle \otimes \cdots \otimes |n_N\rangle$$

Now, imagine we have two functions represented as MPS.

- $C_{n_1,\ldots,n_N} = A^{n_1} A^{n_2} \cdots A^{n_N}$ (χ_A — bond dimension)
- $D_{m_1,\ldots,m_N} = B^{m_1} B^{m_2} \cdots B^{m_N}$ (χ_B — bond dimension)

Although the hadamard product is conceptually simple, for example for rank-2 tensors:

$$A = \begin{pmatrix} 2 & -1 \\ 3 & 5 \end{pmatrix}, B = \begin{pmatrix} -2 & 7 \\ 2 & 1 \end{pmatrix} \Rightarrow A \odot B = \begin{pmatrix} -4 & -7 \\ 6 & 5 \end{pmatrix}$$

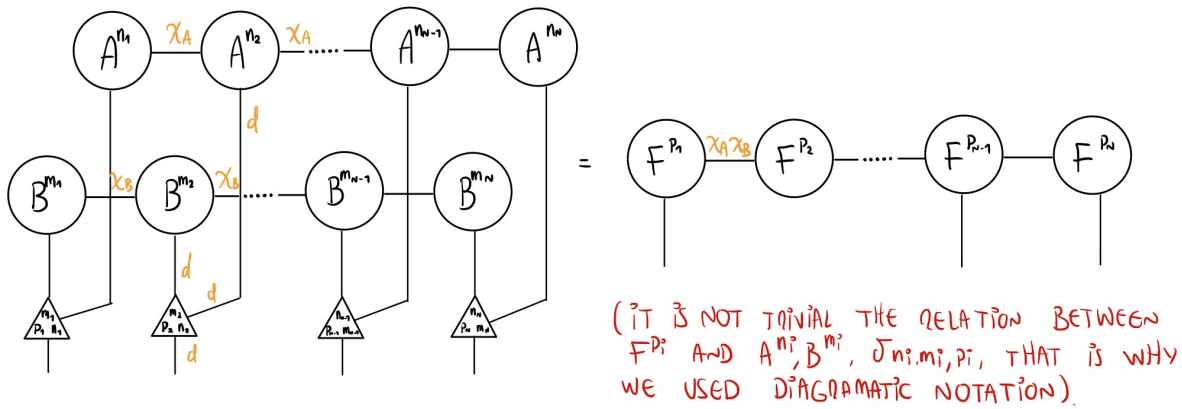
For the MPS case we have

$$F = C \odot D = F_{p_1 p_2 \ldots p_N} = A^{p_1} A^{p_2} \cdots A^{p_N} B^{p_1} B^{p_2} \cdots B^{p_N}.$$

To perform this, we need to define the Kronecker deltas $\delta_{n_j m_j p_j}$, which is just one when $n_j = m_j = p_j$.

$$\therefore \quad F = A^{n_1} A^{n_2} \cdots A^{n_N} B^{m_1} B^{m_2} \cdots B^{m_N} \delta_{n_1 m_1 p_1} \delta_{n_2 m_2 p_2} \cdots \delta_{n_N m_N p_N}$$

In diagrammatic notation:



The complexity scales as $(\chi_A \chi_B)^2$ and **the bond dimension increases as** $\chi_A \chi_B$.

[239]:
```
function Apply_Hadamard_ψ1_ψ2(ψ1, ψ2, χ, cutoff)
    sites = siteinds(ψ1)
    N = length(sites)
    had = MPO(N)
```

```
    had .= convert(MPO, ϕ1)
    for i = 1:N
        had[i] *= δ(sites[i], sites[i]', sites[i]'')
    end
    had = replaceprime(had, 2 => 0)
    return apply(had, ϕ2; maxdim=χ, cutoff=cutoff)
end
```

[239]: `Apply_Hadamard_ϕ1_ϕ2 (generic function with 1 method)`

To be honest, I have never used this operation in my research. It tends to appear in **non-linear** differential equations, and most of quantum mechanics consists of **linear** differential equations.

Let's take as an example $h(x) = f(x)g(x)$ with $ f(x) = sin(x)$ and $g(x) = xe^{0.25x}$.

[247]:
```
N = 7  # Number of qubits used for encoding the function.
x_values = range(0, 2pi, length=2^N); # Sampling grid points

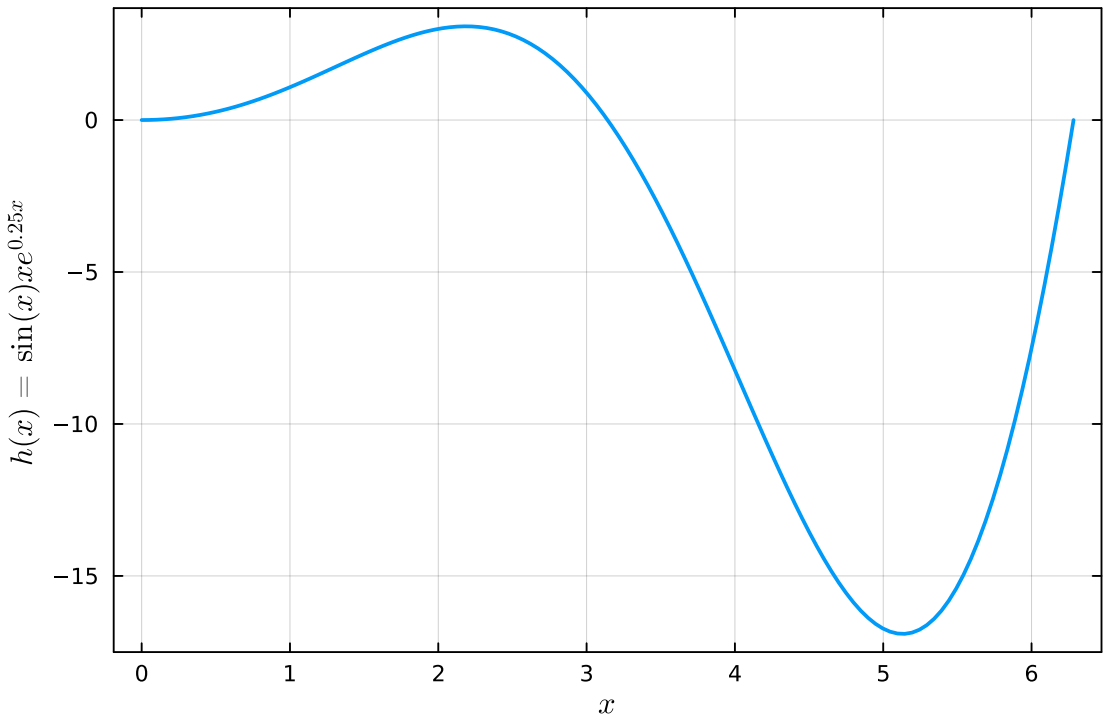
f(x) = sin(x); # First function
y1 = f.(x_values); # Construction f(x)

g(x) = x*exp(0.25*x); # Second function
y2 = g.(x_values); # Construction g(x)

h(x) = f(x)*g(x);
y3 = h.(x_values); # Construction h(x)

plot(x_values, y3, frame = :box, dpi = 200, linewidth = 2.0, label = false,␣
↪xlabel = L"x", ylabel = L"h(x) = \sin (x) x e^{0.25 x}") # Plotting the␣
↪original function
```

[247]:



[250]:
```
sites = siteinds("Qubit", N); # Indices for all sites

cutoff = 1E-8;
χ = 1;

ψ1 = MPS(y1, sites, cutoff=cutoff, maxdim=χ);
ψ2 = MPS(y2, sites, cutoff=cutoff, maxdim=χ);
ψ3 = MPS(y3, sites, cutoff=cutoff, maxdim=χ);

@show linkdims(ψ1);
@show linkdims(ψ2);
@show linkdims(ψ3);


ψ3_Had = Apply_Hadamard_ψ1_ψ2(ψ1, ψ2, χ, cutoff)
@show linkdims(ψ3_Had);
```

```
linkdims(ψ1) = [1, 1, 1, 1, 1, 1]
linkdims(ψ2) = [1, 1, 1, 1, 1, 1]
linkdims(ψ3) = [1, 1, 1, 1, 1, 1]
linkdims(ψ3_Had) = [1, 1, 1, 1, 1, 1]
```

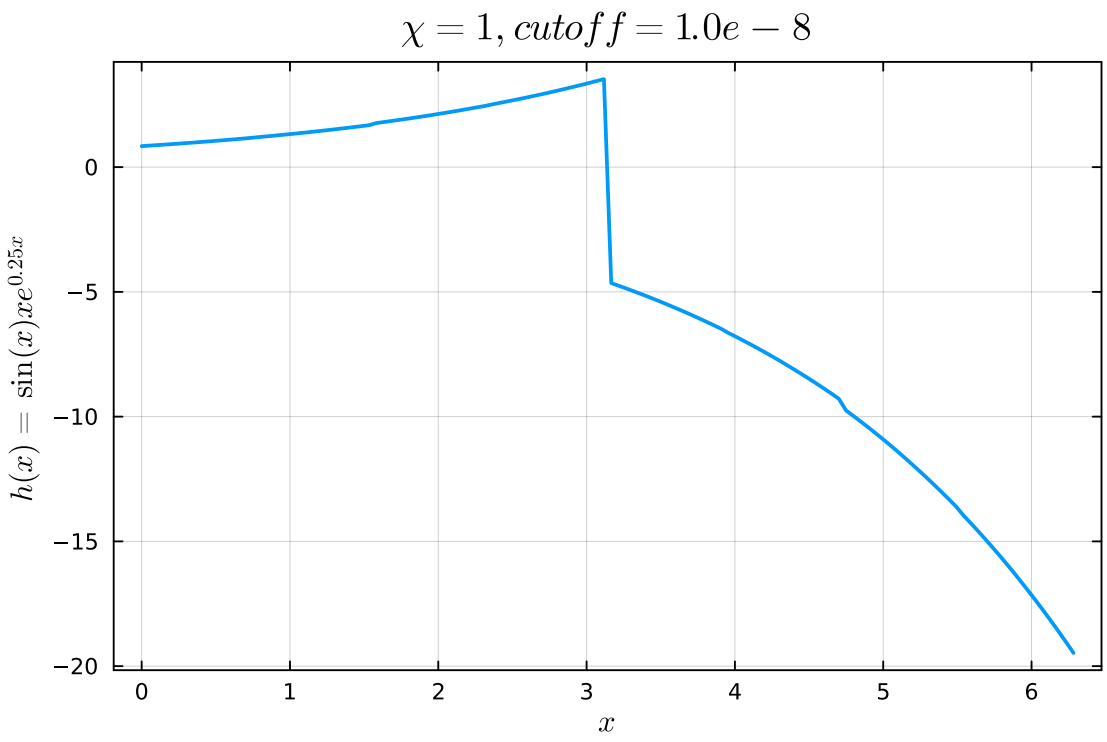
[251]:
```
# Now let's go back to standardJulia array objects to see that we have the same␣
 ↪information.

Big_ψ = contract(ψ3_Had)
comb = combiner(sites)
y_recon = Array(Big_ψ*comb, combinedind(comb)) # Changing the format from␣
 ↪ITensor to Julia array for plotting7

#If we do not want to use the combiner, we could also do this (this is how JJ␣
 ↪does it).
# J_recon = Array(Big_ψ, s) # Changing the format from ITensor to Julia array␣
 ↪for plotting
# y_recon = reshape(J_recon,2^N,1) # Reshaping from N-dimentional tensor to␣
 ↪regular array

plot(x_values, y_recon, frame = :box, dpi = 200, linewidth = 2.0, label = false,␣
 ↪xlabel = L"x", ylabel = L"h(x) = \sin (x) x e^{0.25 x}", title = L"\chi = %$χ,␣
 ↪cutoff = %$cutoff") # Plotting the reconstructed function
```

[251]:



15.16 Exercise: Encode $h(x) = f^2(x)g(x)$ with $ f(x) = \cos(x)$ and $g(x) = \sqrt{x}e^{-0.25x}$ for $x \in [0, 20]$ using 8 qubits.

[190]:
```
N = 8  # Number of sites
x_values = range(0, 20, length=2^N); # Sampling grid points

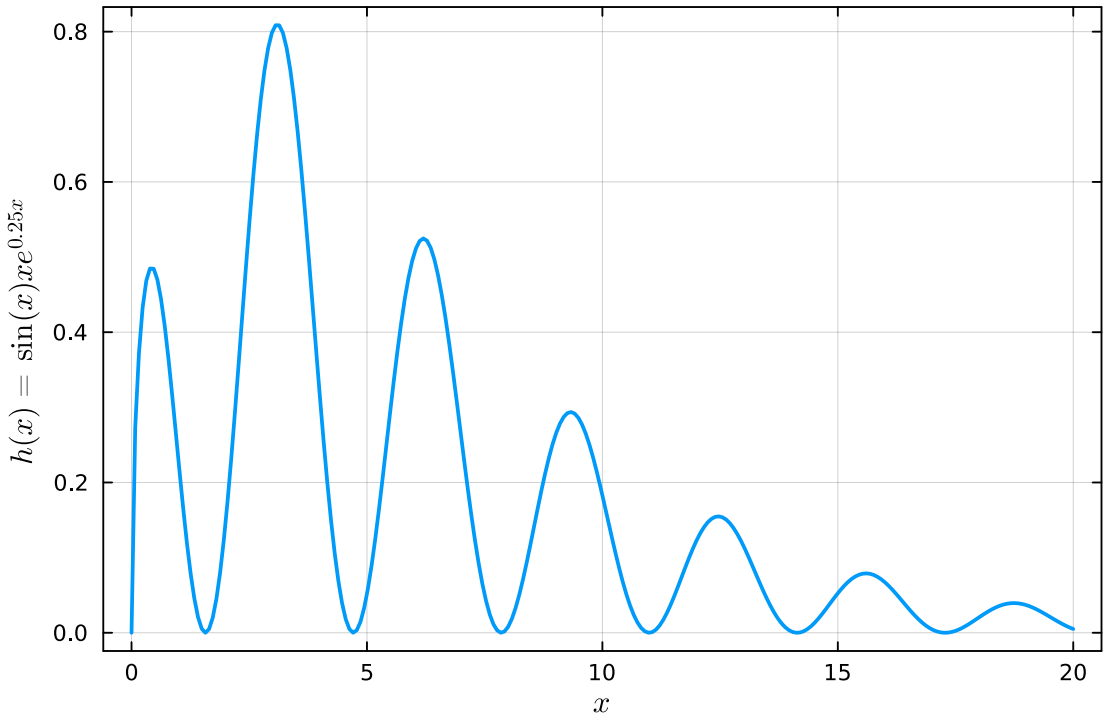
f(x) = cos(x); # First function
y1 = f.(x_values); # Construction f(x)

g(x) = sqrt(x)*exp(-0.25*x); # Second function
y2 = g.(x_values); # Construction g(x)

h(x) = f(x)*f(x)*g(x);
y3 = h.(x_values); # Construction h(x)

plot(x_values, y3, frame = :box, dpi = 200, linewidth = 2.0, label = false,␣
↪xlabel = L"x", ylabel = L"h(x) = \sin (x) x e^{0.25 x}") # Plotting the␣
↪original function
```

[190]:



[191]:
```
sites = siteinds("Qubit", N); # Indices for all sites

cutoff = 1E-8;
χ = 10;
```

```
ψ1 = MPS(y1, sites, cutoff=cutoff, maxdim=χ);
ψ2 = MPS(y2, sites, cutoff=cutoff, maxdim=χ);
ψ3 = MPS(y3, sites, cutoff=cutoff, maxdim=χ);

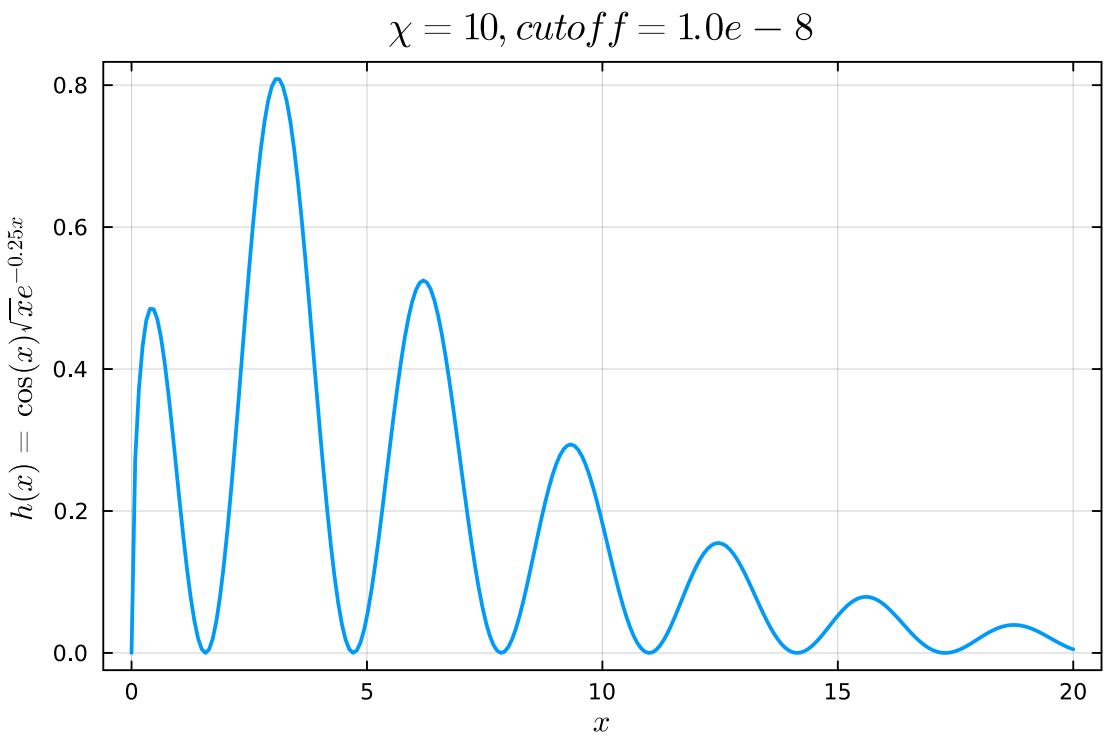
@show linkdims(ψ1);
@show linkdims(ψ2);
@show linkdims(ψ3);

ψ_Had1 = Apply_Hadamard_ψ1_ψ2(ψ1, ψ1, χ, cutoff)
ψ_Had2 = Apply_Hadamard_ψ1_ψ2(ψ_Had1, ψ2, χ, cutoff)

Big_ψ = contract(ψ_Had2)
comb = combiner(sites)
y_recon = Array(Big_ψ*comb, combinedind(comb)) # Changing the format from␣
 ↪ITensor to Julia array for plotting
plot(x_values, y_recon, frame = :box, dpi = 200, linewidth = 2.0, label = false,␣
 ↪xlabel = L"x", ylabel = L"h(x) = \cos (x) \sqrt{x} e^{-0.25 x}", title =␣
 ↪L"\chi = %$χ, cutoff = %$cutoff") # Plotting the reconstructed function
```

```
linkdims(ψ1) = [2, 2, 2, 2, 2, 2, 2]
linkdims(ψ2) = [2, 3, 4, 4, 3, 3, 2]
linkdims(ψ3) = [2, 4, 5, 5, 6, 4, 2]
```

[191]:



16. GPU speed-up

Everything we've built so far — random ITensors, MPS construction, SVDs, operator applications — has been running on the CPU, where the cubic scaling in χ we just discussed can become a real bottleneck as bond dimensions grow. Since **these operations are fundamentally dense linear algebra (matrix multiplications, SVDs, contractions), they map naturally onto GPU hardware**, which is built precisely to accelerate this kind of computation. The core idea is straightforward: move the relevant tensors onto the GPU, perform the same operations we've already learned — contraction, SVD, apply(), MPS construction — and let the underlying library dispatch to GPU-accelerated linear algebra routines instead of CPU ones, with **no change to the tensor-network logic itself.**

In practice, the most important part is being sure about where your data lives and how much of it fits. GPUs typically have far less memory than a CPU-accessible RAM pool, so an MPS or intermediate tensor that fits comfortably on the CPU can quickly exhaust GPU memory as bond dimensions grow. Crucially, **ITensors does not clean up GPU memory for us** — unlike Julia's CPU garbage collector, which you can mostly rely on passively, GPU memory needs to be actively monitored and freed as the code runs, or you'll run out of memory well before you expect to.

Note: The slow first execution of Julia is independent of GPU and CPU. You need to precompile whatever you want to use on the hardware that you will use.

[243]:
```
# using Pkg
# Pkg.add("CUDA") #You need to install CUDA if you do not have it already.␣
↪Sometimes this can be annoying depending on the drivers of your GPU

using CUDA
```

[244]:
```
CUDA.device() #This should print your GPU. If not, something was wrong in the␣
↪installation.
```

[244]:
```
CuDevice(0): NVIDIA GeForce RTX 4070 Laptop GPU
```

16.1 Moving data to the GPU

In general we just need to use the function `cu()` or `NDTensors.cu()` to define the same ITensor object inside the GPU Memory as a CUArray. The only difference between the two functions is that NDTensors.cu() preserves the element type of the tensors, while cu() converts to single precision. I **ALWAYS** uses `NDTensors.cu()` because **single precision can generate problems in DMRG or TEBD** algorithms: https://itensor.discourse.group/t/tebd-with-gpu-error-with-eigen/1266/5 .

[252]:
```
#woGPU
sites = siteinds("S=1/2",50)
bond_dimension = 200 #The difference between times increases with this value.

A = randomMPS(sites, bond_dimension)
O = randomMPO(sites)

@time inner(A, apply(O, A))
```

```
  14.171078 seconds (63.98 k allocations: 933.278 MiB, 3.73% gc time)
```

[252]:
```
-3.470469394352001e-16
```

[253]:
```
#wGPU
A = NDTensors.cu(A)
O = NDTensors.cu(O)

@time inner(A, apply(O, A))
```

```
  0.628477 seconds (165.77 k allocations: 21.780 MiB, 41 lock conflicts)
```

[253]:
```
-3.470469394420194e-16
```

16.2 How to supervise memory?

[155]:
```
CUDA.pool_status() #This is how you check the memory status of your GPU. In the␣
↪past, it was memory_status(). The name can vary with the version of CUDA.
```

```
Effective GPU memory usage: 22.64% (1.810 GiB/7.996 GiB)
Memory pool usage: 64.622 MiB (672.000 MiB reserved)
```

16.3 How to clean memory?

Since ITensors doesn't automatically free GPU memory for us, we need to explicitly trigger cleanup when memory usage grows too large — especially in loops where many intermediate tensors get created and discarded (e.g. repeated SVDs, sweeps, or gate applications).

- `GC.gc(true)` forces Julia's garbage collector to run a full, deep collection, freeing any CPU-side objects (including wrapper objects pointing to GPU arrays) that are no longer referenced. This is thorough but comparatively slow, since it walks the entire object graph rather than just recently allocated memory.
- `CUDA.reclaim()` asks the CUDA memory pool to release GPU memory that has already been marked as free (typically by a prior garbage collection) back to the device. This is a much faster, GPU-side-only operation, but it only reclaims memory that Julia's GC has already flagged as unreachable — it doesn't do the reachability analysis itself.

Let's see this by repeating the previous operation; **without cleaning at some point, the GPU will be full, and the code will be slower.**

[156]:
```
sites = siteinds("S=1/2",50)
bond_dimension = 100 #The difference between times increases with this value.

A = randomMPS(sites, bond_dimension)
O = randomMPO(sites)
A = NDTensors.cu(A)
O = NDTensors.cu(O)
```

[157]:
```
for i = 1:20
    @time begin
        @show i
        inner(A, apply(O, A))
        CUDA.pool_status()

        #Cleaning:
        GC.gc(true) #Deep cleaning. Takes more time
        CUDA.reclaim() #Fast cleaning. Runs very fast.
        CUDA.pool_status()
    end
end
```

[158]:
```
GC.gc(true) #Deep cleaning. Takes more time
CUDA.reclaim() #Fast cleaning. Runs very fast.
```

